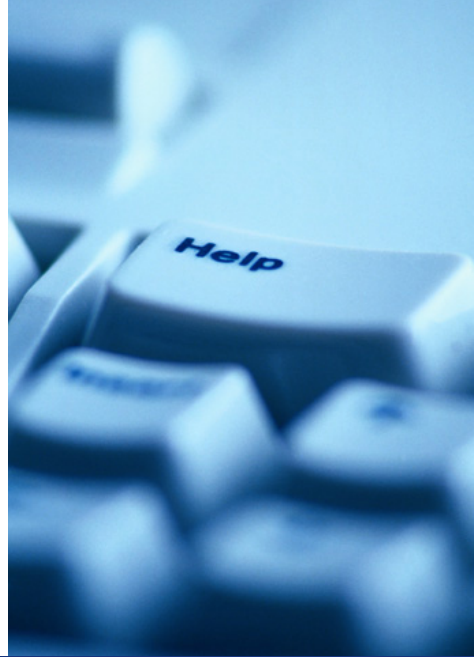




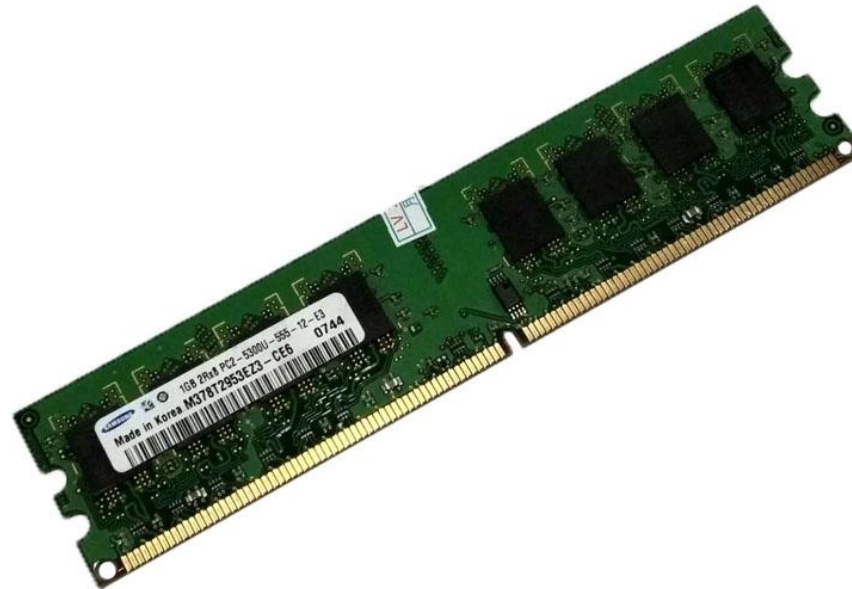
操作系统原理

Operating Systems Principles

陈鹏飞
计算机学院



第十讲 — 虚拟内存管理





目标

- 定义虚拟内存；
- 讨论虚拟内存的优点；
- 解释请求调页、页面置换算法和页帧分配等概念；
- 讨论工作集模型原理；
- 讨论共享内存和内存映射文件之间的关系；
- 讨论如何管理内核内存；

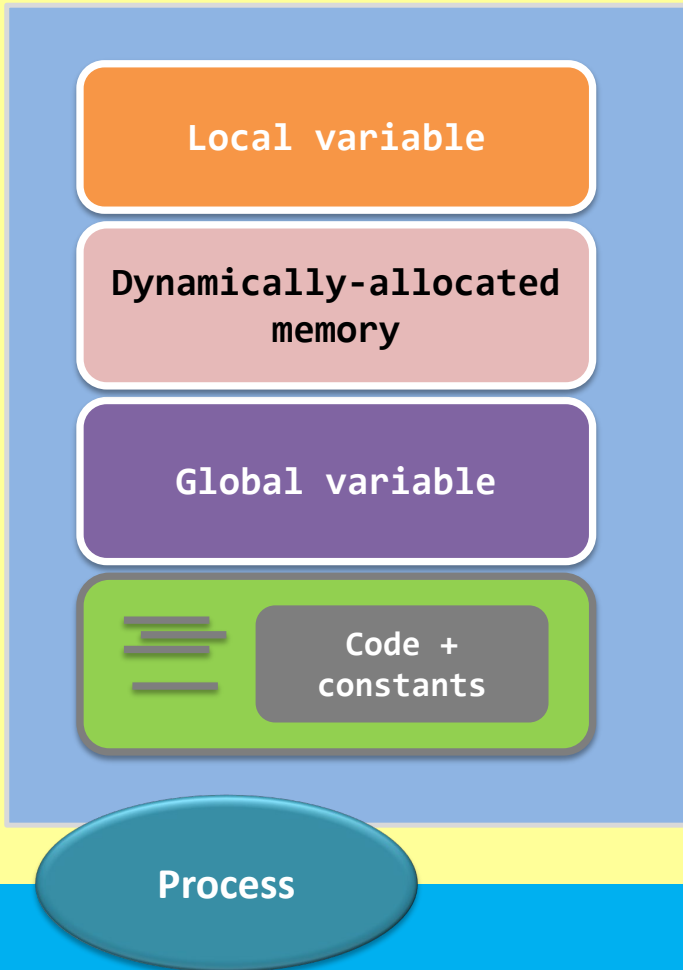


背景

- ❖ 代码需要在内存中执行，但很少使用整个程序
 - 错误代码、异常例程、大型数据结构；
- ❖ 不需要同时使用整个程序代码；
- ❖ 考虑执行部分加载程序，可以带来很多好处：
 - 程序不再受物理内存限制；
 - 每个程序在运行时占用更少内存→同时运行更多程序
 - 在不增加响应时间或周转时间的情况下提高CPU利用率和吞吐量；
 - 将程序加载或交换到内存所需的I/O更少→每个用户程序运行更快；



Memory management



How to use the addresses to access the memory device?

How do multiple process share the same physical memory device?

How to support large process?

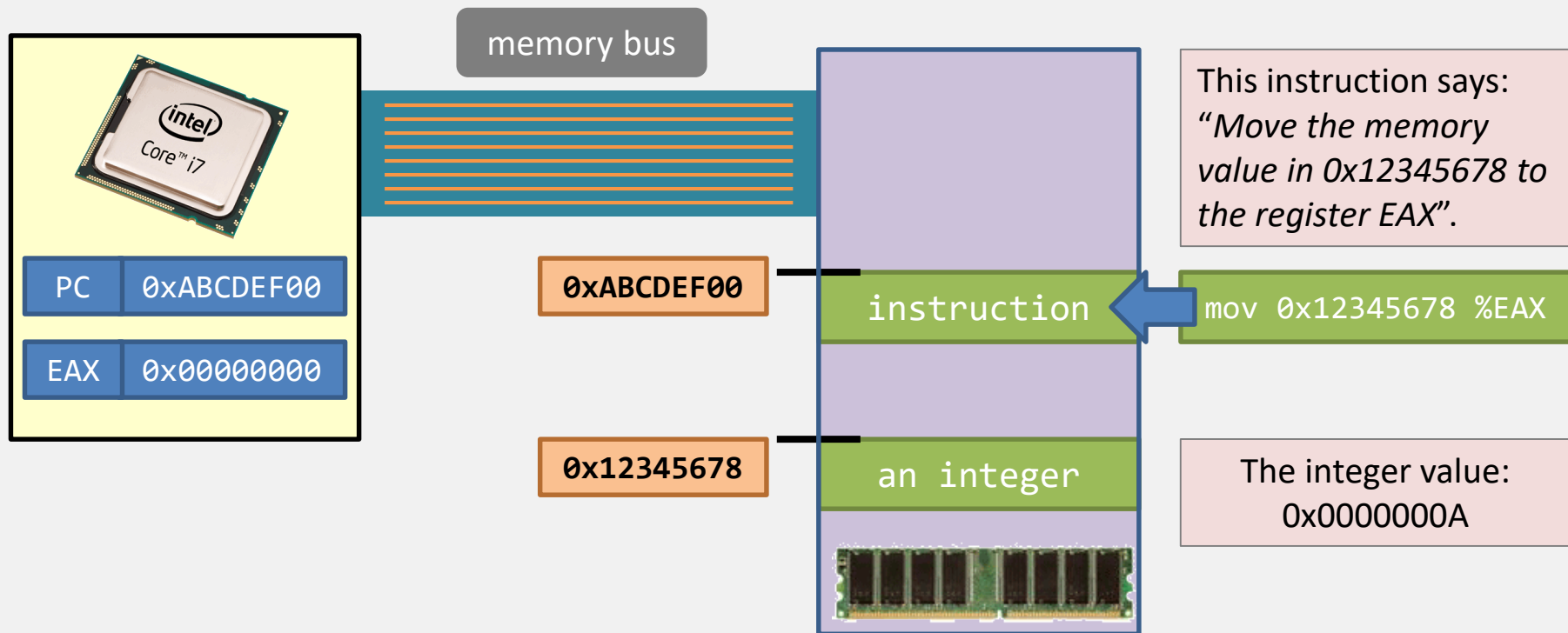
How does the CPU read what it wants from the memory device?

.....

The kernel and the hardware are doing lots of managements...

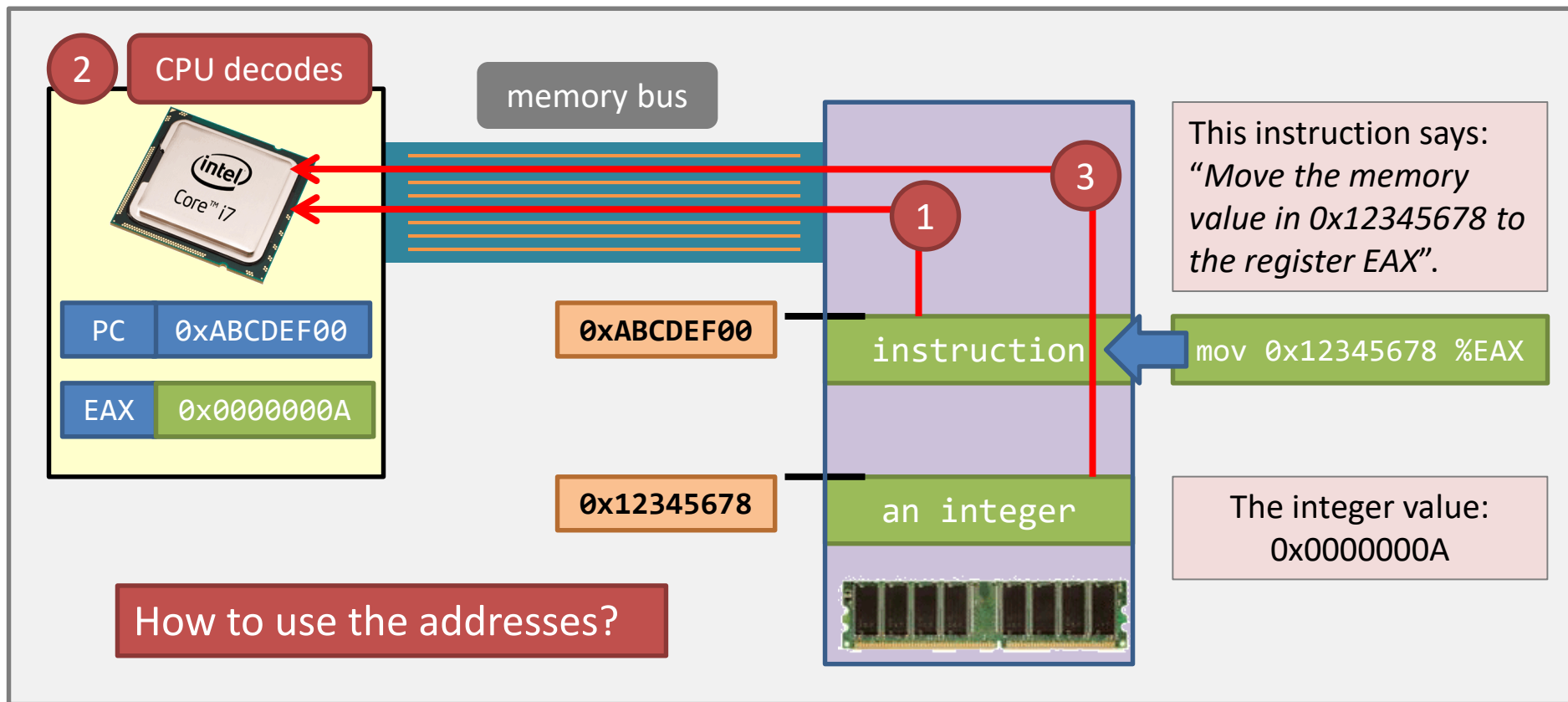


- Let's review the “fetch-decode-execute” cycle!





- Let's review the “fetch-decode-execute” cycle!





```
int main(void) {  
    int pid;  
    pid = fork();  
    printf("PID %d: %p.\n", getpid(), &pid);  
    if(pid)  
        wait(NULL);  
    return 0;  
}
```

```
$ ./same_addr  
PID 1234: 0xbfe85e0c.  
PID 1235: 0xbfe85e0c.  
$_
```

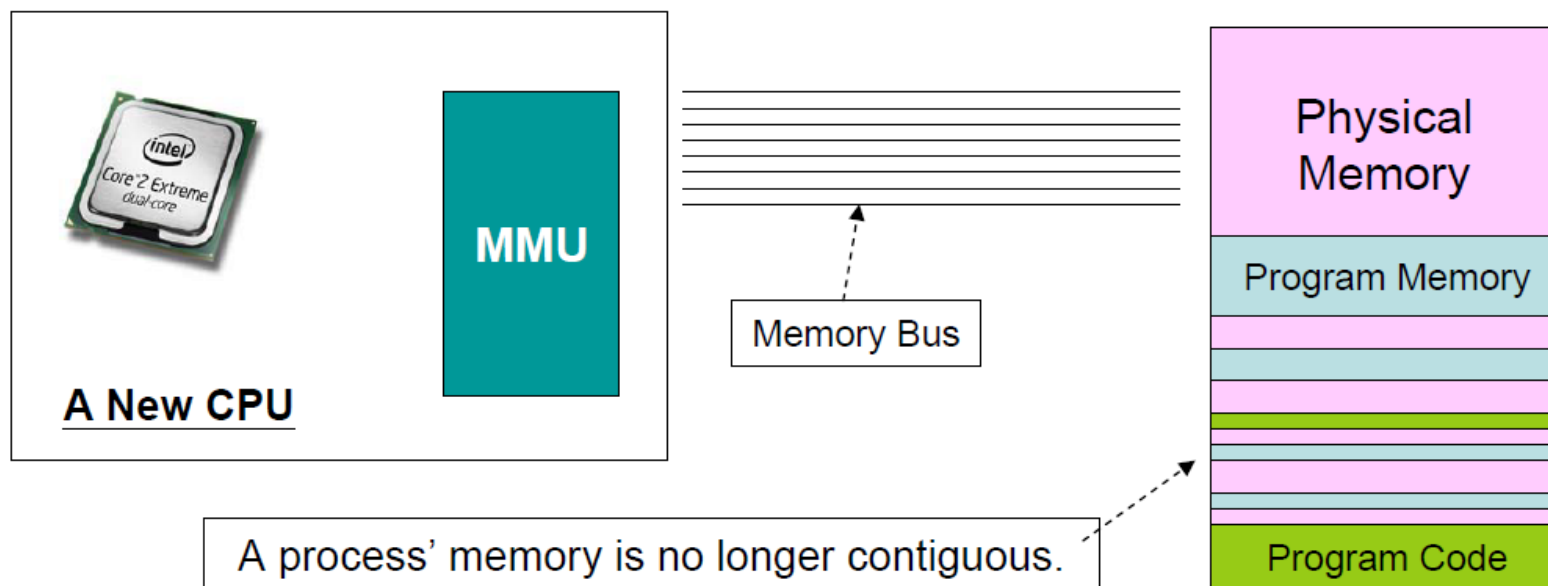
- Can you guess the result?
 - Two **different processes**, the **same variable name**, carry **different values**
 - Use the **same address**! (What? How COME?!)
- Well, what is the meaning of a memory address?!
 - Logical address: **virtual memory**
 - Address translation needed (logical/virtual->physical)
 - Why we use virtual memory??

Demo



虚拟内存的支持

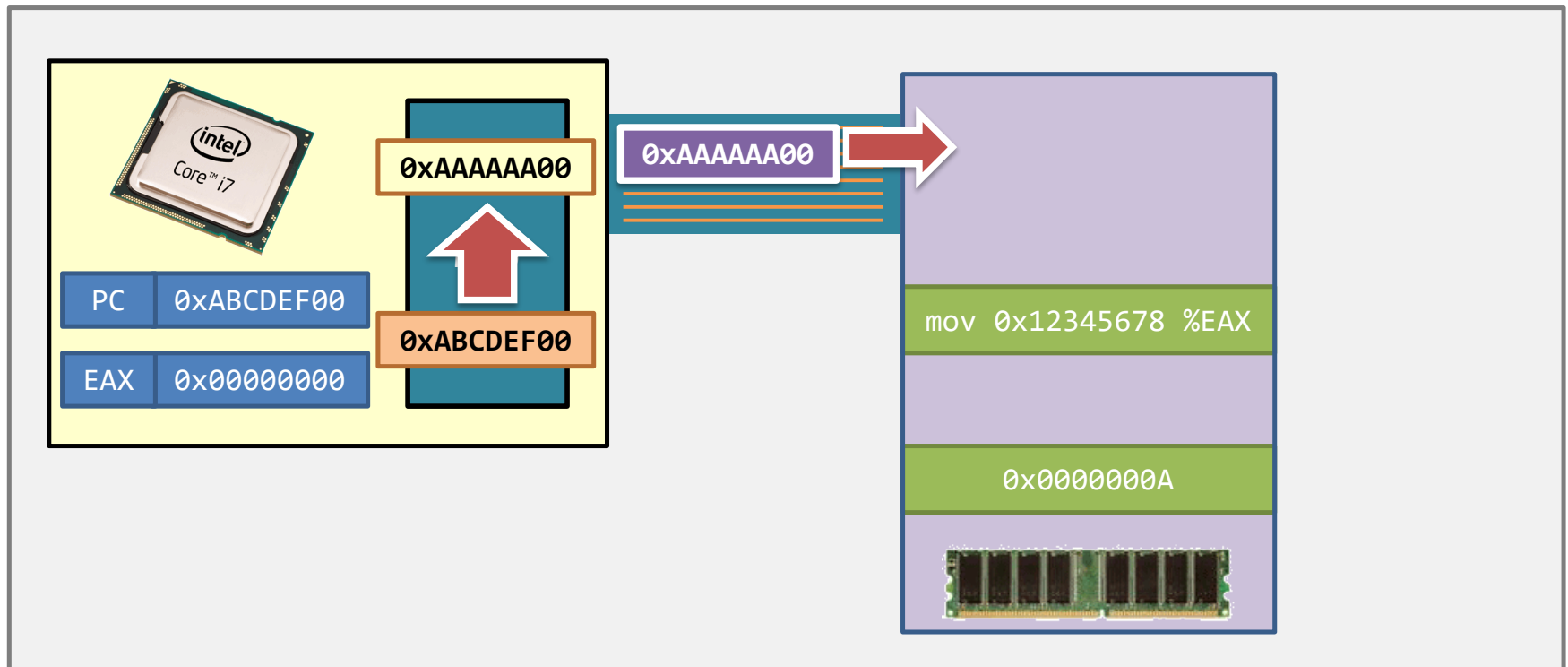
- The new design of the CPU includes a new module: the **memory management unit (MMU)**.
 - MMU is designed to perform address translation.
 - The **MMU** is an **on-CPU device**.





虚拟内存

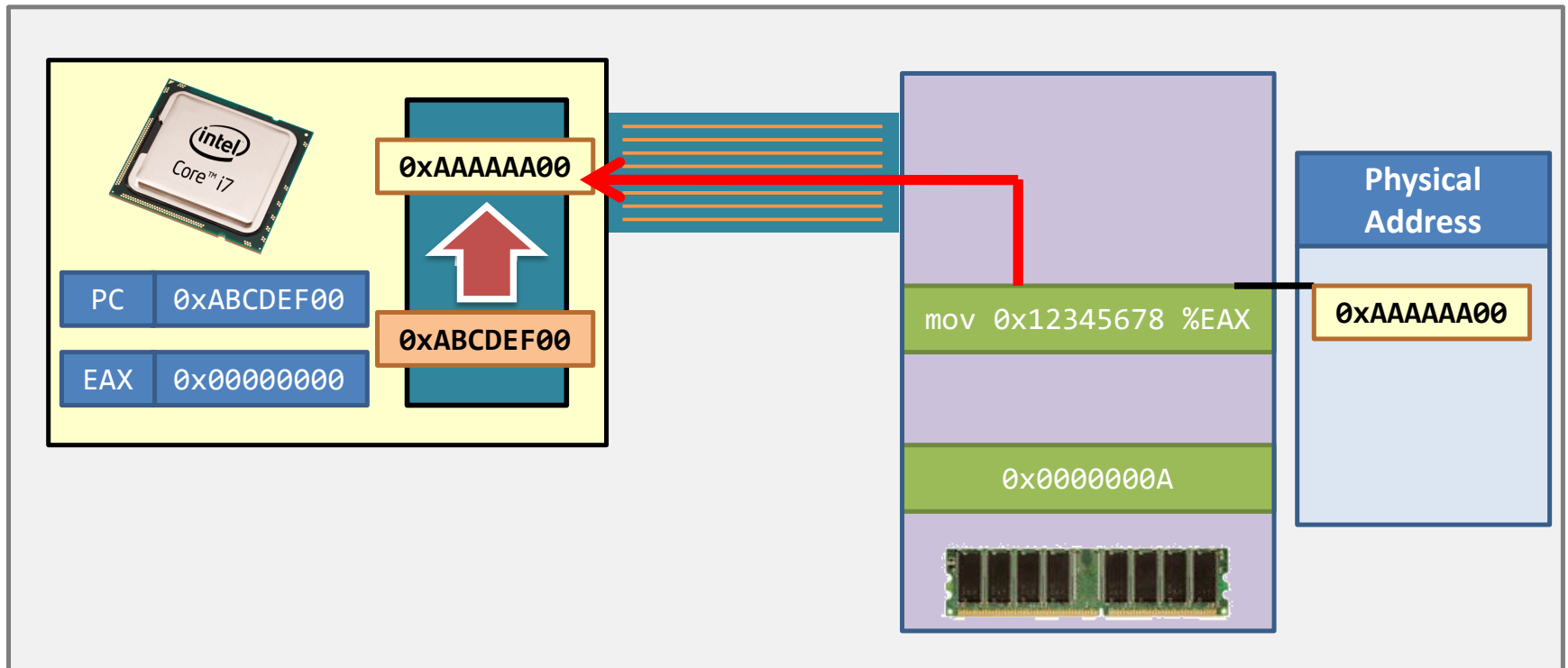
- Step 1. When CPU wants to fetch an instruction, the virtual address is sent to MMU and is translated into a physical address.





虚拟内存

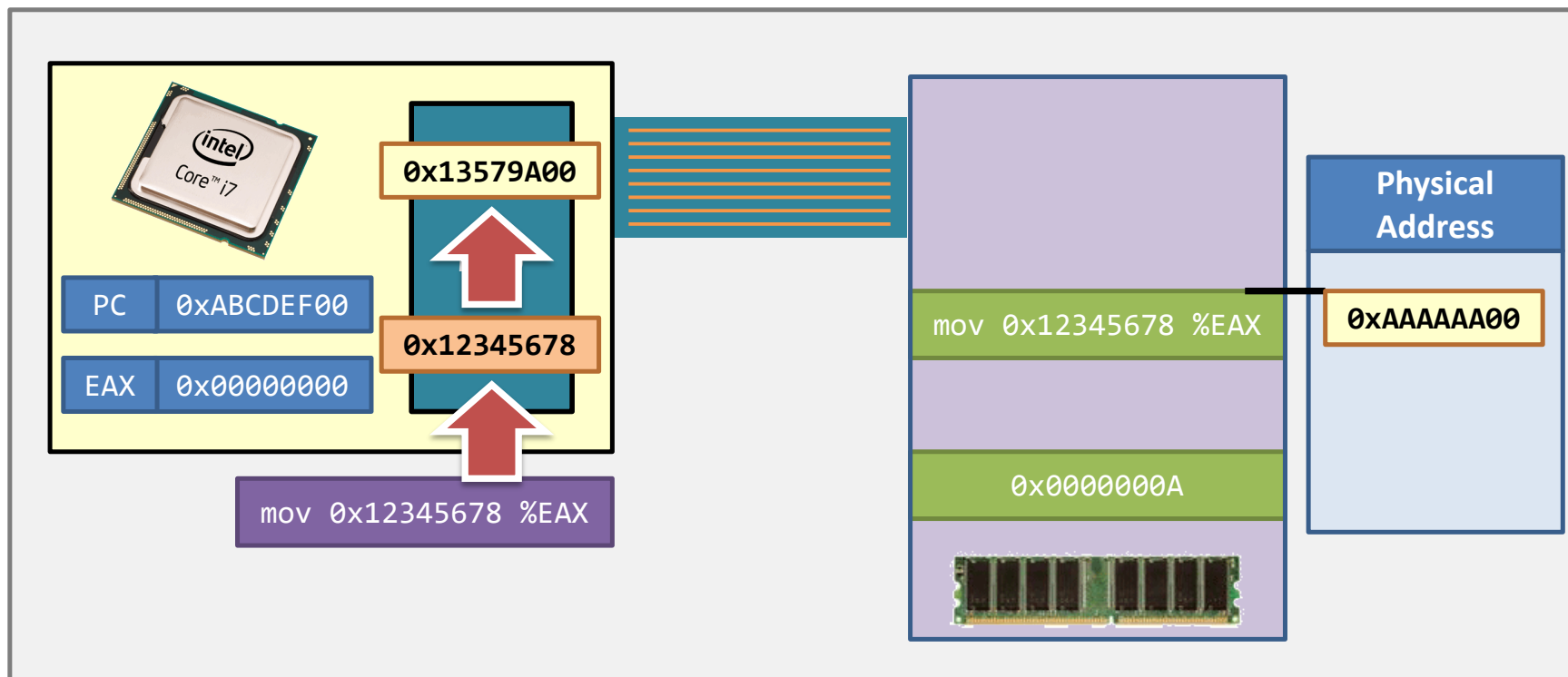
- Step 2. The memory returns the instruction addressed in physical address.





虚拟内存

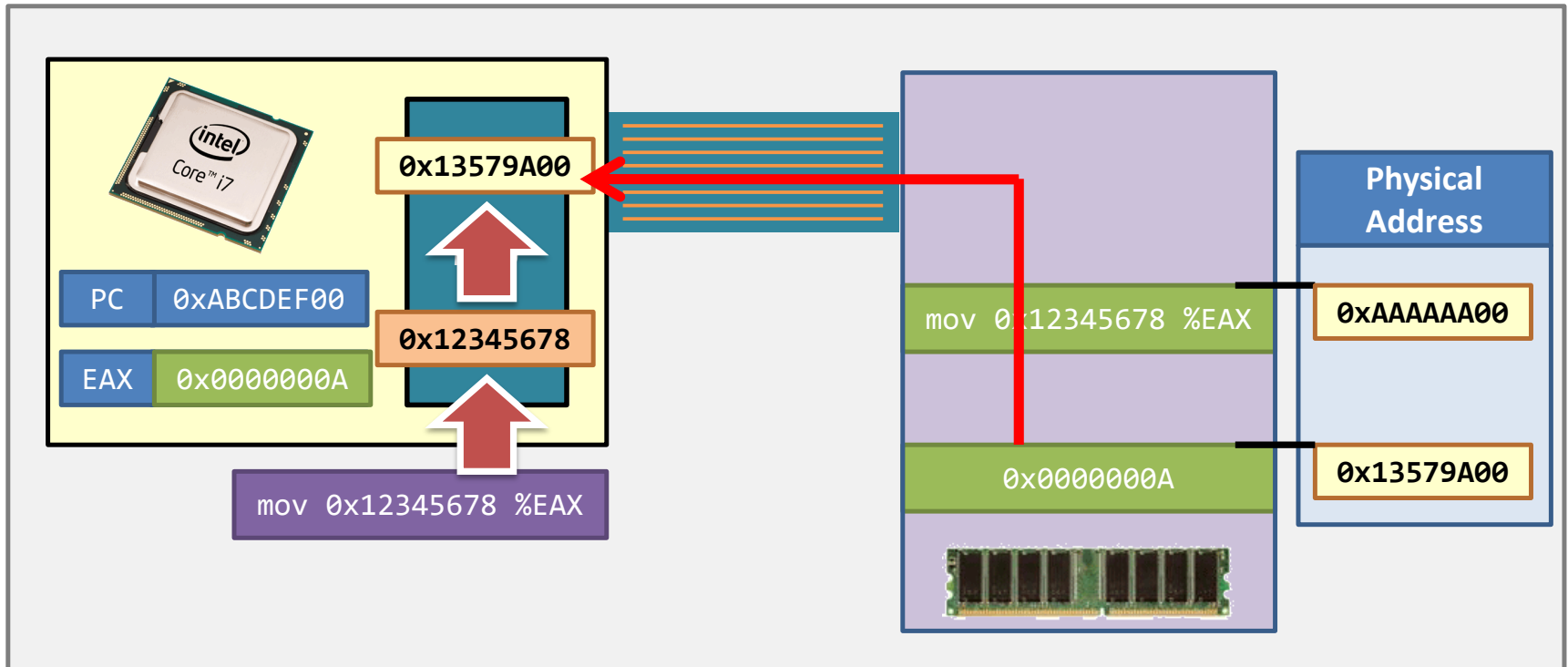
- Step 3. The CPU decodes the instruction.
 - An instruction **always stores virtual addresses**, but not physical addresses.





虚拟内存

- Step 4. With the help of the MMU, the target memory is retrieved.





虚拟内存定义

- ❖ 虚拟内存 – 将用户逻辑内存与物理内存分离
 - 只有部分程序需要在内存中才能执行
 - 因此，逻辑地址空间可以比物理地址空间大得多
 - 允许多个进程共享地址空间
 - 允许更高效的进程创建
 - 更多程序同时运行
 - 加载或交换进程所需的I/O更少；



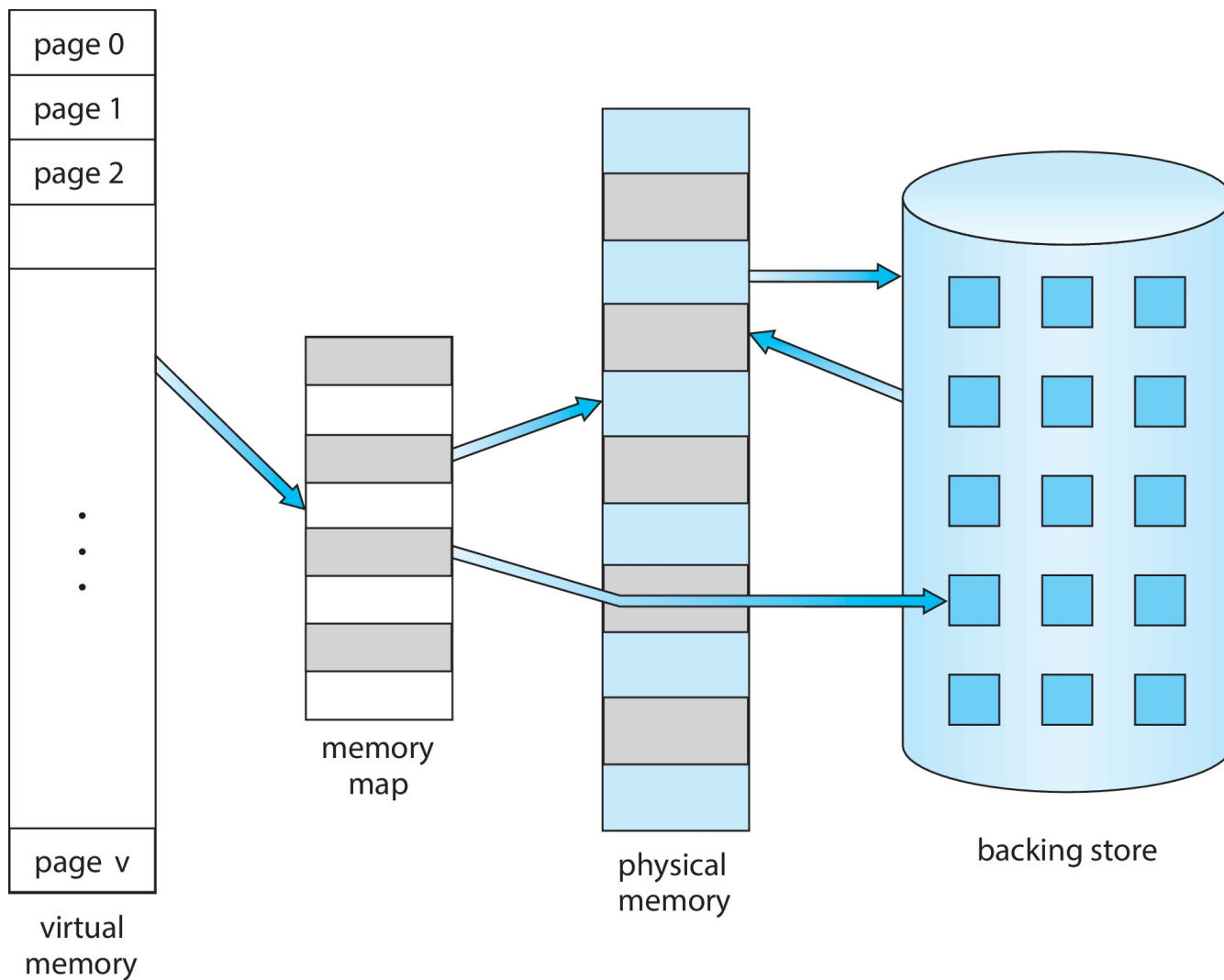
虚拟内存定义

- ❖ 虚拟地址空间 - 进程如何存储在内存中的逻辑视图
 - 通常从地址0开始, 连续地址直到空间结束
 - 同时, 物理内存按照页帧组织, 分配给进程的物理帧可以不连续;
 - MMU必须将逻辑页映射到物理页帧;

- ❖ 虚拟内存可以通过以下方式实现:
 - 请求分页
 - 按需分段



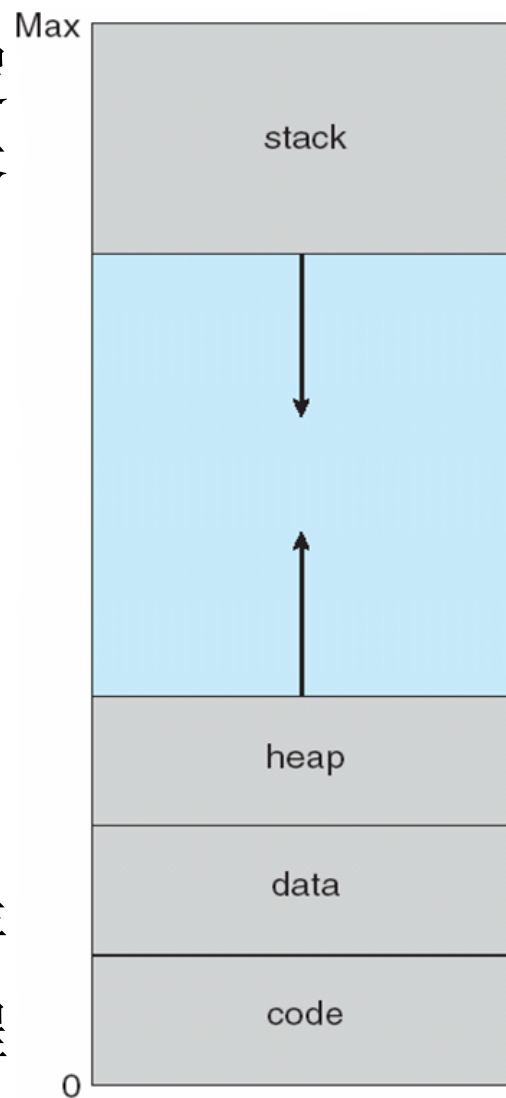
远大于物理内存的虚拟内存





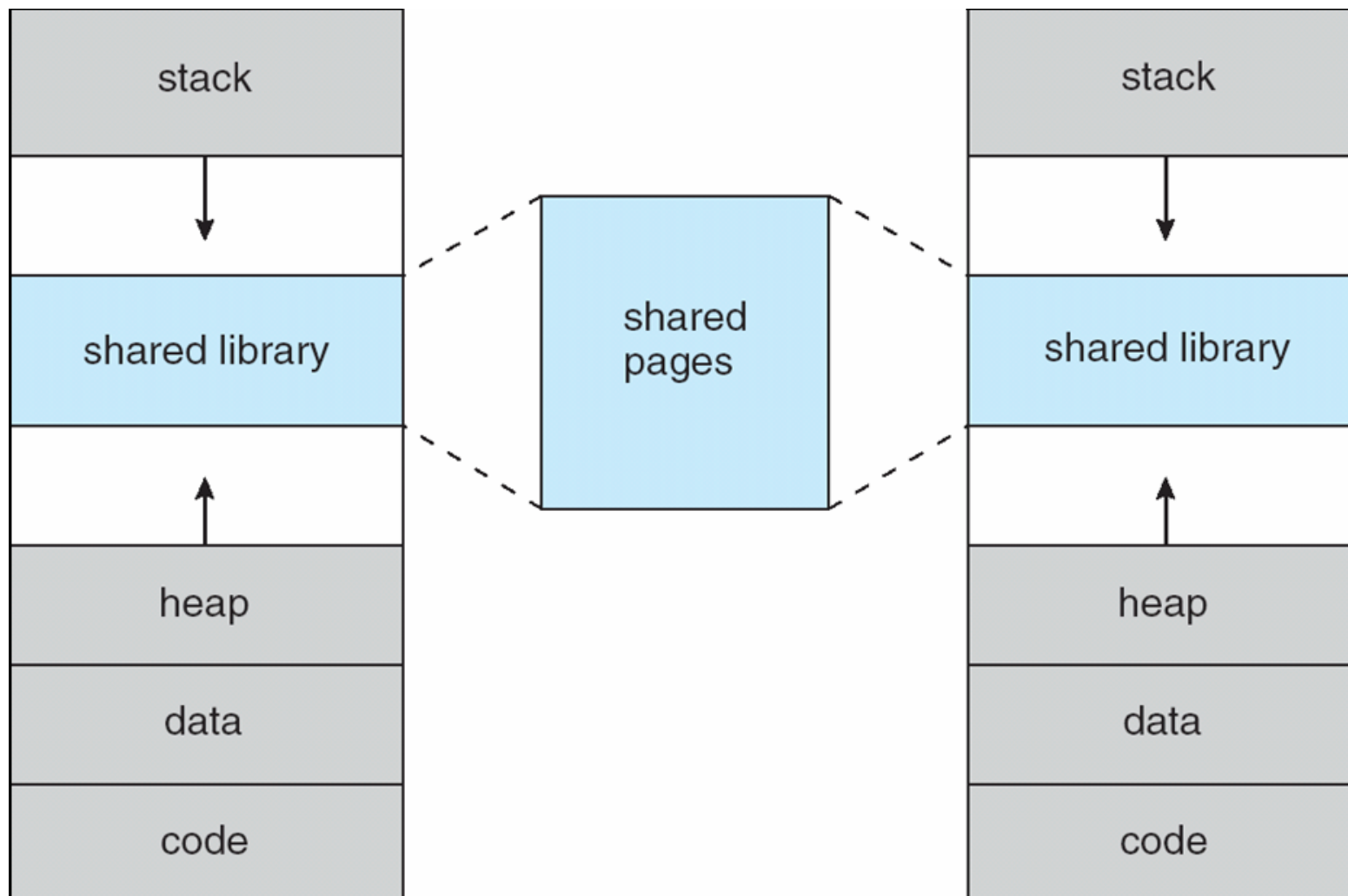
虚拟地址空间

- 通常为堆栈设计逻辑地址空间，使其从最大逻辑地址开始，并在堆“向上”时“向下”增长
 - 最大限度地利用地址空间
 - 两者之间未使用的地址空间为空洞
 - ▶ 只有在堆栈生长时才需要实际的物理页；
 - ▶ 启用具有孔的稀疏地址空间以供增长、动态链接库等。
- 通过映射到虚拟地址空间共享的系统库
- 通过将页读写映射到虚拟地址空间来共享内存
- 在fork（）期间可以共享页面，从而加快进程创建；





使用虚拟地址的共享库





请求 (按需) 分页

- ❖ 可以在加载时将整个进程放入内存或者部分放入内存

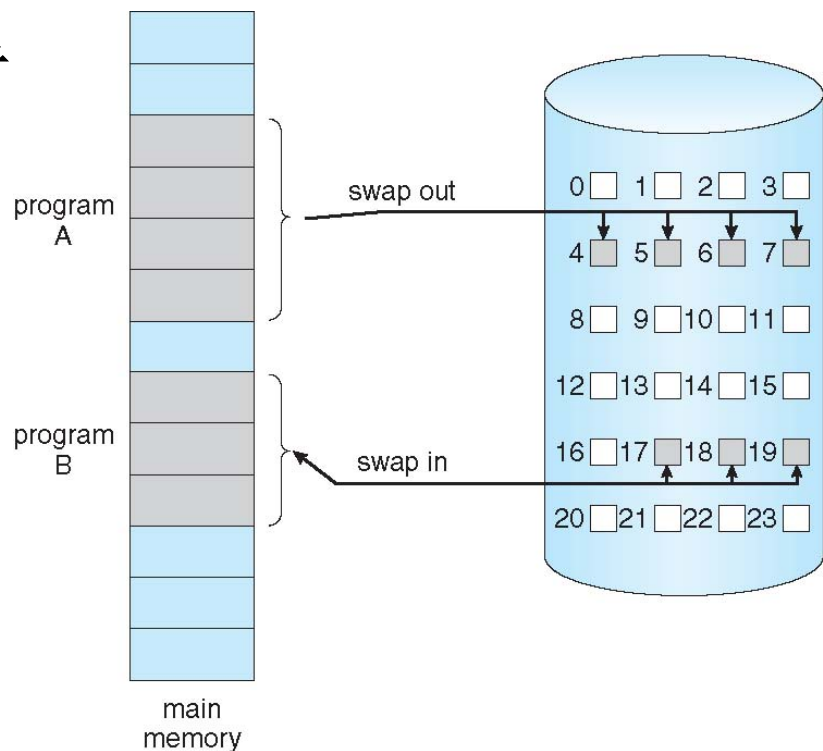
- 所需的I/O更少, 没有不必要的I/O;
- 所需内存更少
- 更快的响应;
- 更支持多用户;

- ❖ 类似于带交换的分页系统 (需要硬件支持);

- 页面是必需的 引用它
- 无效引用 终止
- 不在内存中 调入内存

- ❖ 惰性交换程序 - 除非需要页, 否则永远不要将页交换到内存中

- 处理页面的交换程序是“调页程序”





请求（按需）分页

- ❖ 通过交换，调页程序在再次交换之前猜测将使用哪些页面
- ❖ 调页程序只将这些要使用的页面带入内存
- ❖ 如何确定这组页面？
 - 需要新的MMU功能来实现请求分页
- ❖ 如果所需的页已经驻留在内存中
 - 与非请求分页没有区别
- ❖ 如果需要页但不是内存驻留页面时
 - 需要检测页面并将其从磁盘加载到内存中
 - 不需要改变程序行为；
 - 无需程序员更改代码；



有效-无效位

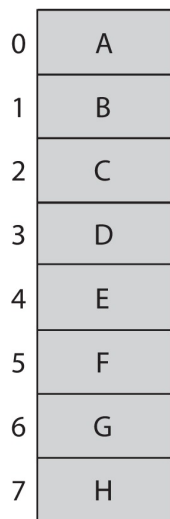
- ❖ 每个页表条目都关联一个有效-无效位 (v表示在内存中 - 内存驻留, i表示不在内存中)
- ❖ 初始有效 - 所有条目上的无效位均设置为i
- ❖ 页表快照示例:
- ❖ 在MMU地址转换期间, 页表条目中如果有效无效位为i, 发生页面错误 (缺页)

Frame #	valid-invalid bit
	v
	v
	v
	i
...	
	i
	i

page table



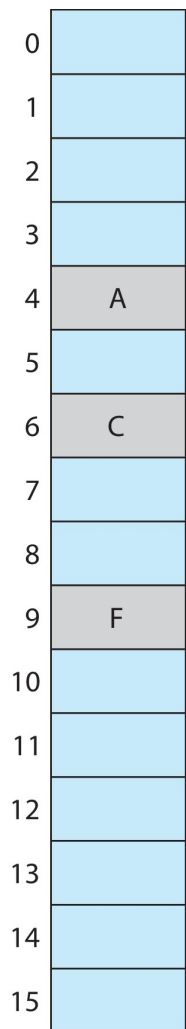
某些页不在主存的情况



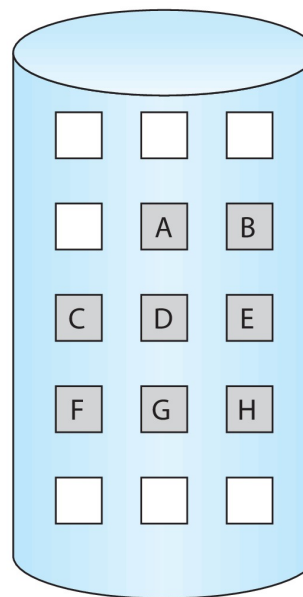
logical memory

valid-invalid bit		
frame		bit
0	4	v
1		i
2	6	v
3		i
4		i
5	9	v
6		i
7		i

page table



physical memory



backing store

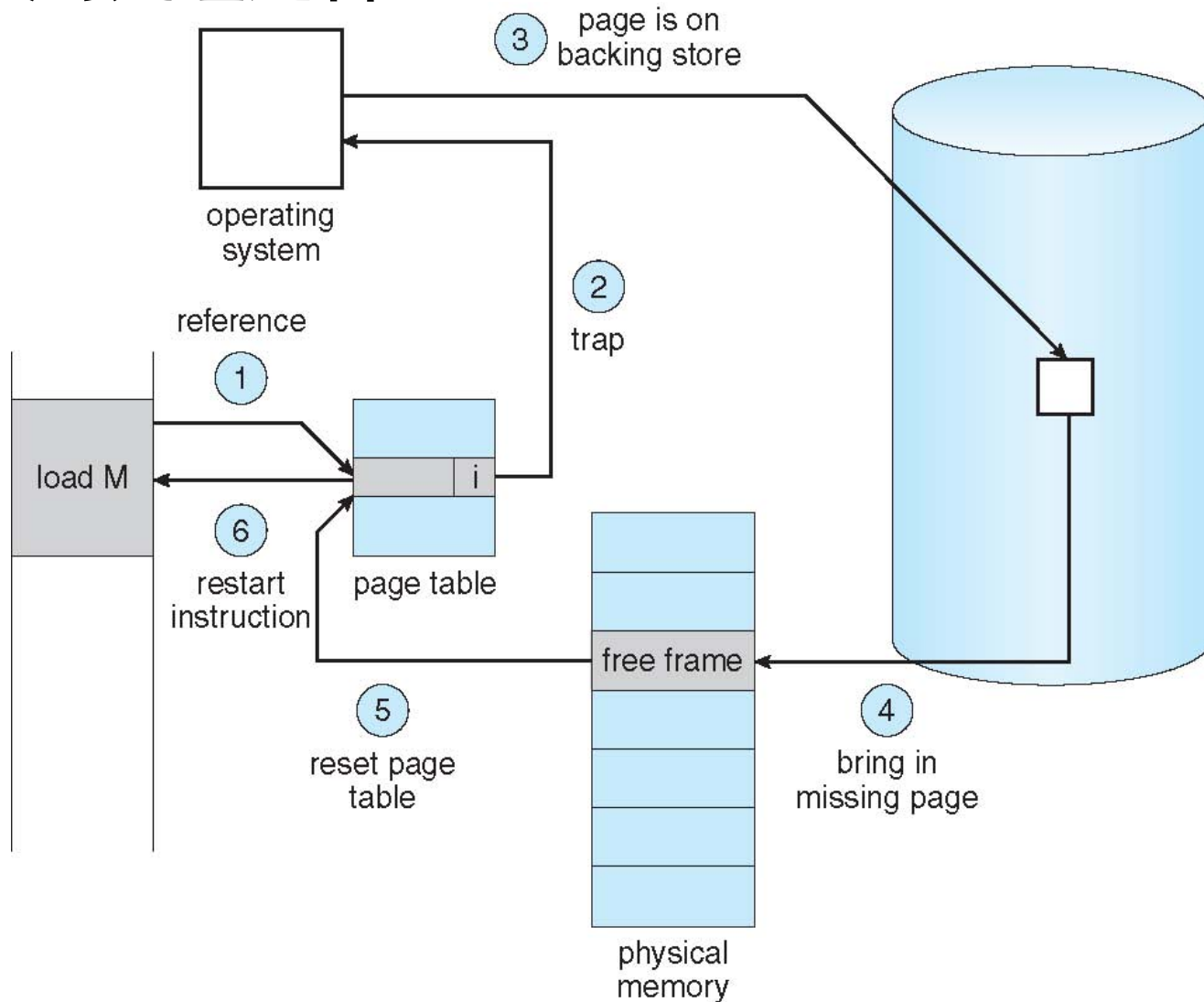


处理缺页

1. 检查进程的內部表，通常与PCB一起保存，以确定内存引用是否有效；
2. 如果无效，那么终止进程。如果引用有效但是未调入页面，那么发生缺页中断；
 - 页面错误（缺页）
3. 找到空闲帧；
4. 通过磁盘调度操作将页面交换到刚分配的帧中；
5. 当磁盘操作读取完成后，重置内部表和页表，以指示当前页面在内存中；
6. 重新启动导致缺页的指令，该进程能够访问所需的页；



缺页错误的处理逻辑





请求分页几个方面的问题

❖ 极端情况 - 启动进程时内存中没有页面

- 操作系统将指令指针设置为进程的第一条指令，非内存驻留→页面错误
- 进程其他页面的访问也是一样；
- 纯请求分页；

❖ 实际上，给定的指令可以访问多页→多页错误

- 考虑从内存中将2个数加和并将结果存储回内存的指令的读取和解码，可能产生多个缺页错误；
- 产生性能痛点，幸运的是因为局部引用的存在，使得请求分页具有较为合理的性能；

❖ 请求分页所需的硬件支持

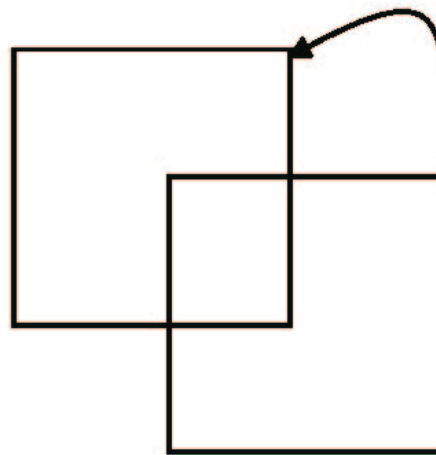
- 具有有效/无效位的页表
- 辅助内存（带交换空间的交换设备）
- 指令重启，在缺页错误后能够重启任何指令；



指令重启

- ❖ 简单重启指令如具有三个地址的指令ADD;
- ❖ **Consider an instruction that could access several different locations**

- Block move (块移动)



- Auto increment/decrement location
- Restart the whole operation?
 - What if source and destination overlap?
 - (源块可能被修改, 不能简单重启指令)



指令重启

❖ 指令操作地址重叠问题的解决方案:

- 微代码计算试图访问源块和目的块的两端，如果出现缺页错误，那么在这一步出现，完成缺页逻辑后，执行移动操作；
- 使用临时寄存器来保存覆盖位置的值，如果有缺页错误，则在陷阱发生之前，所有旧值都将写回到内存中。该动作是将内存恢复到指令启动之前的状态，这样能够重复执行该指令；



空闲帧列表

- ❖ 当出现页面错误时，操作系统必须将所需页面从辅助存储器带入主存。
- ❖ 大多数操作系统都维护一个空闲帧列表——一个用于满足此类请求的空闲帧池。



- ❖ 操作系统通常使用一种称为按需零填充 (zero-fill-on-demand) 的技术来分配空闲帧，即在分配前将帧的内容清零。
- ❖ 当系统启动时，所有可用内存都会放在空闲帧列表中。



请求分页的性能

❖ Three major activities

- Service the interrupt – careful coding means just several hundred instructions needed
- Read the page – lots of time
- Restart the process – again just a small amount of time

❖ Page Fault Rate $0 \leq p \leq 1$

- if $p = 0$ no page faults
- if $p = 1$, every reference is a fault

❖ Effective Access Time (EAT)

$$\begin{aligned} \text{EAT} = & (1 - p) \times \text{memory access} \\ & + p (\text{page fault overhead} \\ & + \text{swap page out} \\ & + \text{swap page in}) \end{aligned}$$



缺页错误导致的一系列动作

1. 操作系统陷入
2. 保存用户寄存器和进程状态
3. 确定中断是否是缺页错误
4. 检查页面引用是否合法，并确定页面在磁盘上的位置；
5. 向空闲帧发出从磁盘读取的命令：
 - a) 在此设备的队列中等待，直到读取请求得到服务；
 - b) 等待设备寻道和/或延迟时间；
 - c) 开始将页面传输到空闲帧；



缺页错误导致的一系列动作

6. 等待时，将CPU分配给其他用户
7. 从磁盘I/O子系统接收中断（I/O完成）
8. 为其他用户保存寄存器和进程状态
9. 确定中断来自磁盘；
10. 更正页表和其他表以显示页现在在内存中；
11. 等待CPU再次调度到此进程；
12. 还原用户寄存器、进程状态和新页表，然后恢复中断的指令



请求分页的性能

- ❖ 内存访问时间=200纳秒
- ❖ 平均页面错误服务时间=8毫秒

$$\text{EAT} = (1-p) \times 200 + p \times (8 \text{ 毫秒})$$

$$= (1 - p \times 200 + p \times 8000000)$$

$$= 200 + p \times 7999800$$

- ❖ 如果1000次访问中有一次导致页面错误，则
 $\text{EAT} = 8.2 \text{ 微秒}$ 。

这是40倍的减速！！

- ❖ 如果希望性能下降<10%

- $220 > 200 + 7999800 \times p$
- $20 > 7999800 \times p$
- $p < 0.0000025$
- <每400000次内存访问中出现一页错误



请求分页性能优化

- ❖ 交换空间I/O比文件系统I/O快，即使在同一设备上也是如此
 - 交换分配在更大的块中，比文件系统所需的管理更少；
- ❖ 在进程加载时将整个进程映像复制到交换空间
 - 然后在交换空间内执行调页；
 - 在旧的BSD Unix中使用；
- ❖ 从磁盘上的程序二进制文件请求分页，但在释放帧时将页面直接放弃（被覆盖），而不是分页调出；
 - 在Solaris和当前BSD中使用
 - 仍然需要写入以交换空间
 - 与文件不关联的页面（如栈和堆）-匿名内存
 - 在内存中修改但尚未写回文件系统的页面；
- ❖ 移动系统
 - 通常不支持交换
 - 相反，从文件系统请求页面并回收只读页面（如代码）

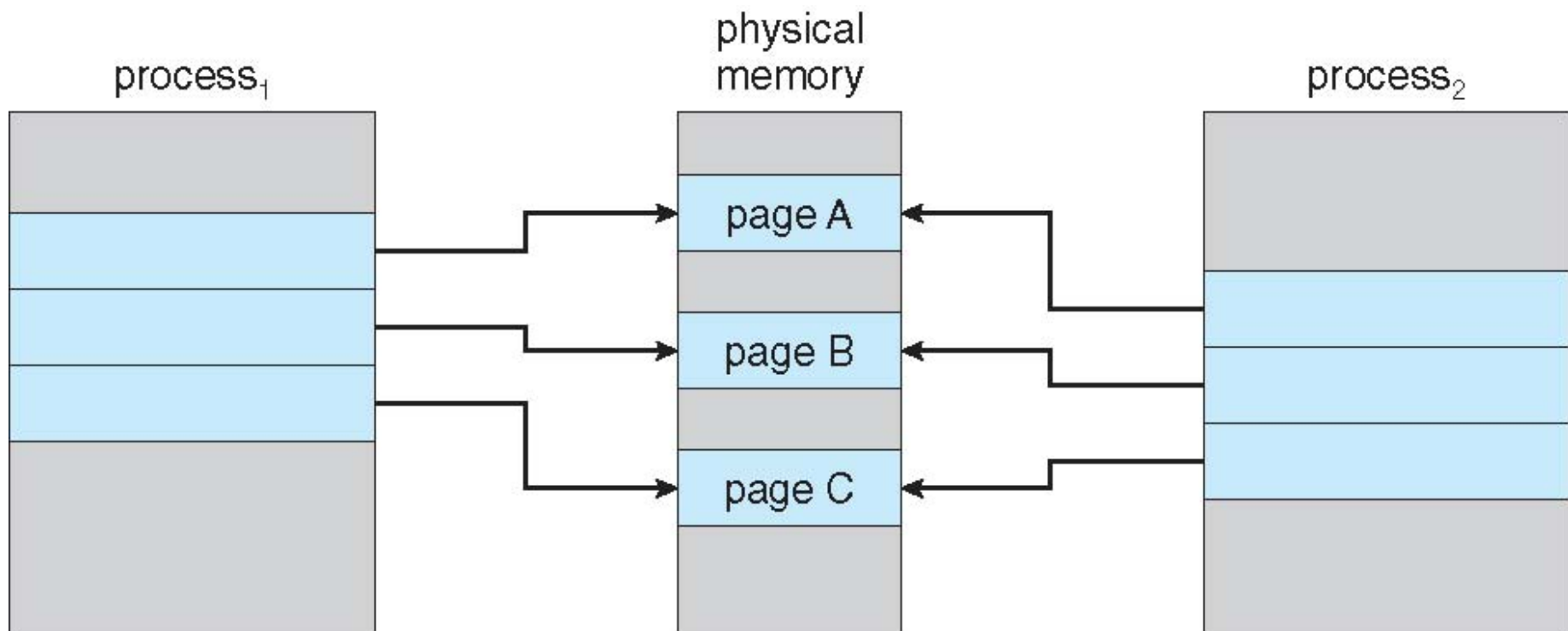


写时复制

- ❖ 写时复制 (CoW) 允许父进程和子进程最初共享内存中的相同页面
 - 如果任一进程修改了共享页面, 则该页面被复制
- ❖ CoW允许更高效的进程创建 (fork), 因为只复制修改过的页面
- ❖ 通常, 空闲页面是从按需零填充页面池中分配的
 - 池应始终具有用于快速请求页面执行的空闲帧
 - 不希望必须释放一个帧以及其他处理页面错误
 - 为什么在分配页面之前将其归零? (清除之前的内容)
- ❖ 基于fork () 系统调用vfork () 变体, 父进程挂起, 子进程使用父进程的地址空间, 不采用写时复制, 因此, 父进程可以看到子进程的修改;
 - 设计为具有子调用exec () 的情况;
 - 不需要复制内存页面, 效率很高;

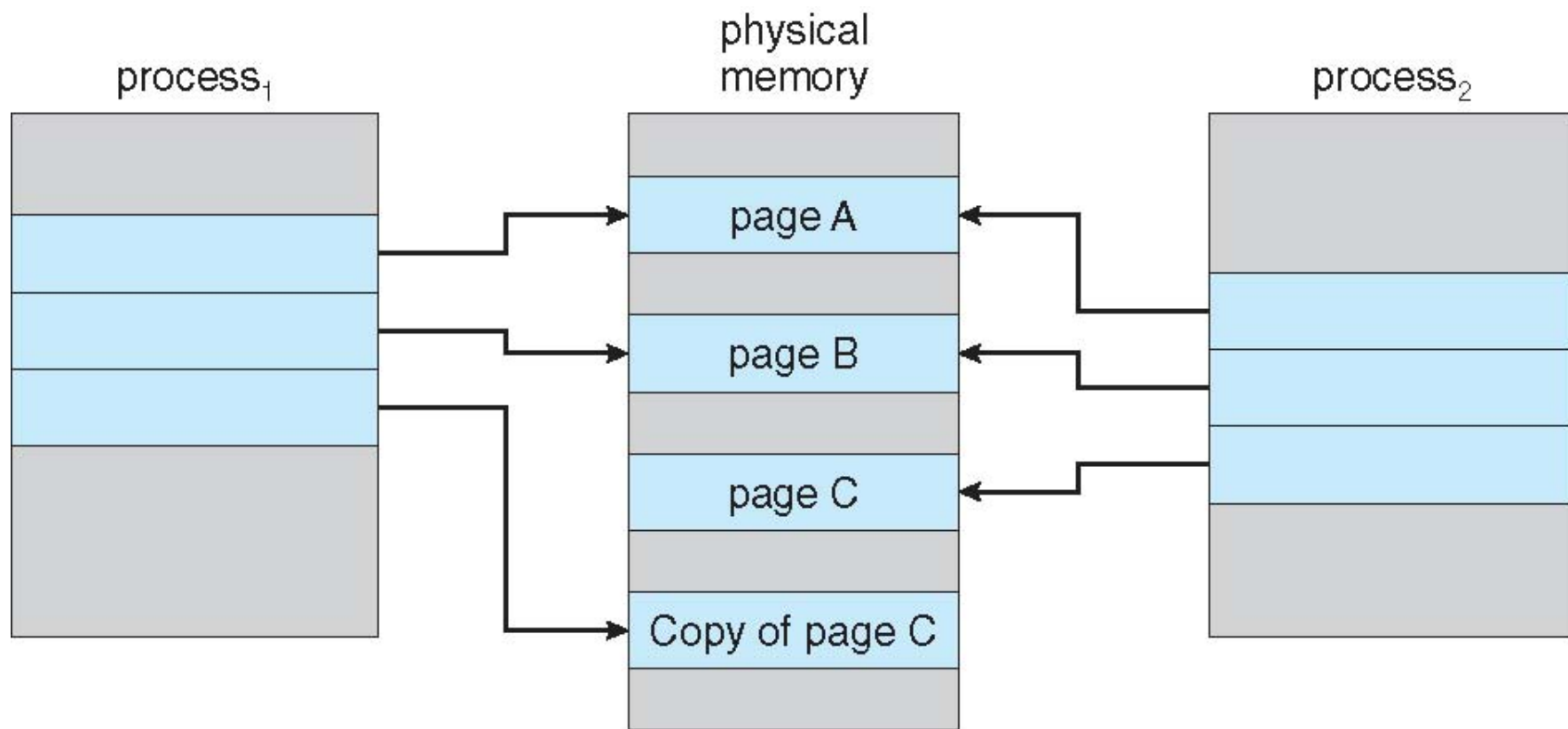


进程修改C页面之前





进程修改页面C之后





如果没有空闲帧时

- ❖ Used up by process pages
- ❖ Also in demand from the kernel, I/O buffers, etc
- ❖ Over allocating (过度分配) ;
- ❖ How much to allocate to each?
- ❖ Page replacement – find some page in memory, but not really in use, page it out
 - Algorithm – terminate? swap out? replace the page?
 - Performance – **want an algorithm which will result in minimum number of page faults**
- ❖ Same page may be brought into memory several times

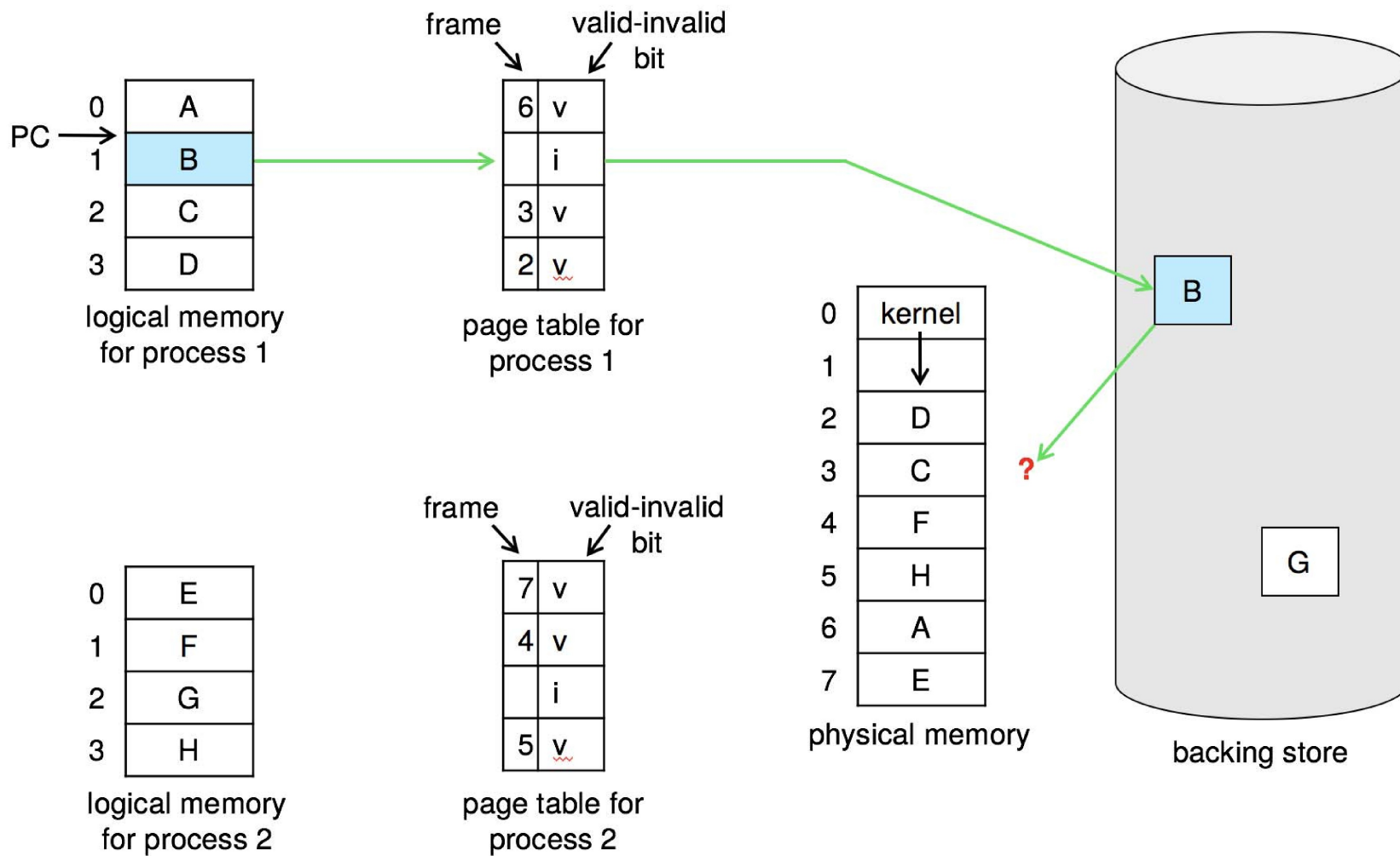


页面置换

- ❖ 通过修改页面故障服务例程以包括页面替换来防止内存过度分配；
- ❖ 使用修改（脏）位来减少页面传输的开销—只有修改过的页面才会写入磁盘；
- ❖ 页面替换完成了逻辑内存和物理内存之间的分离——可以在较小的物理内存上提供较大的虚拟内存；



页面置换





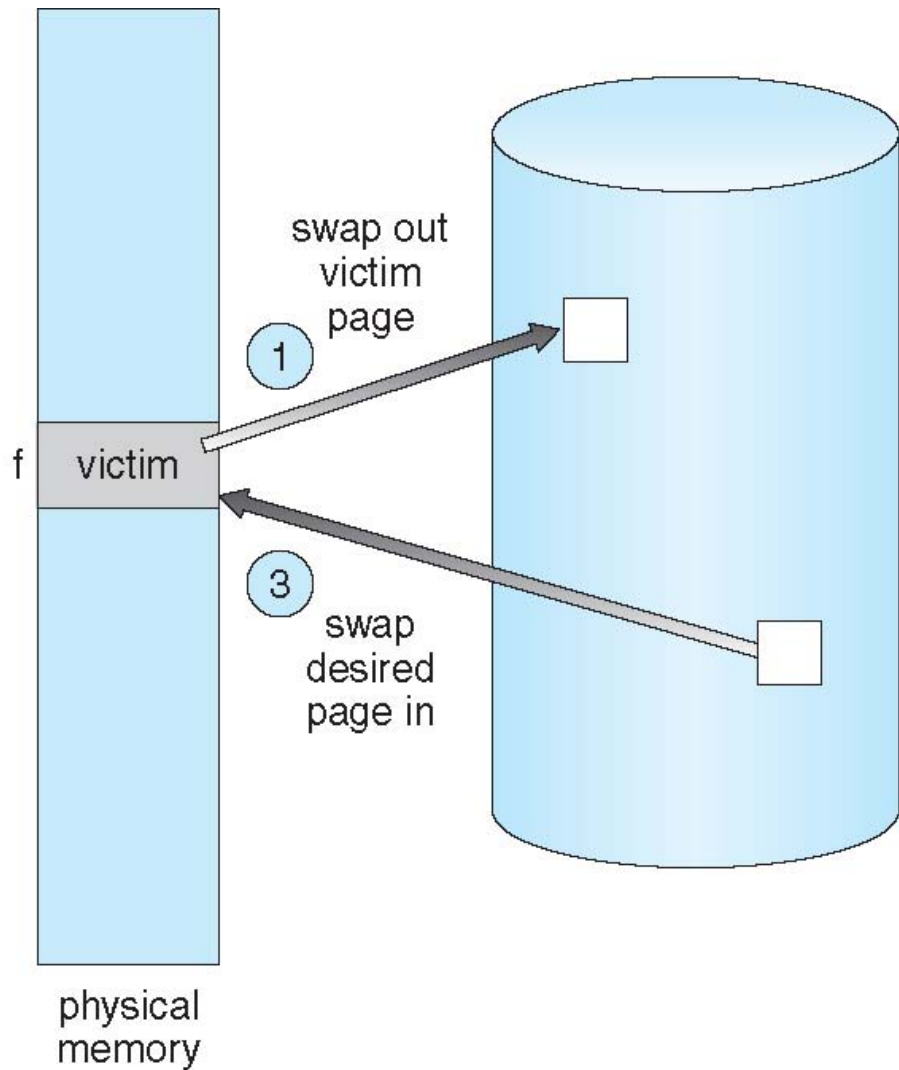
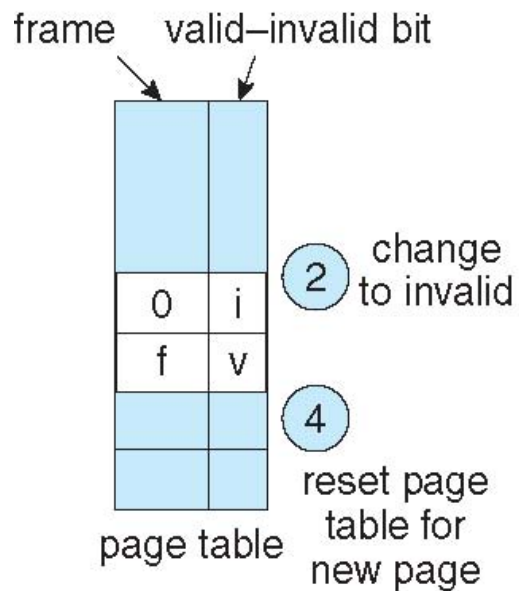
基本的页面置换

1. Find the location of the desired page on disk
2. Find a free frame:
 - If there is a free frame, use it
 - If there is no free frame, use **a page replacement** algorithm to select a victim frame
 - Write victim frame to disk if dirty
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Continue the process by restarting the instruction that caused the trap

Note now potentially 2 page transfers for page fault – increasing EAT
采用 Dirty Bit 脏位可减少这种开销;



页面置换





页面和帧置换算法

❖ 帧分配算法

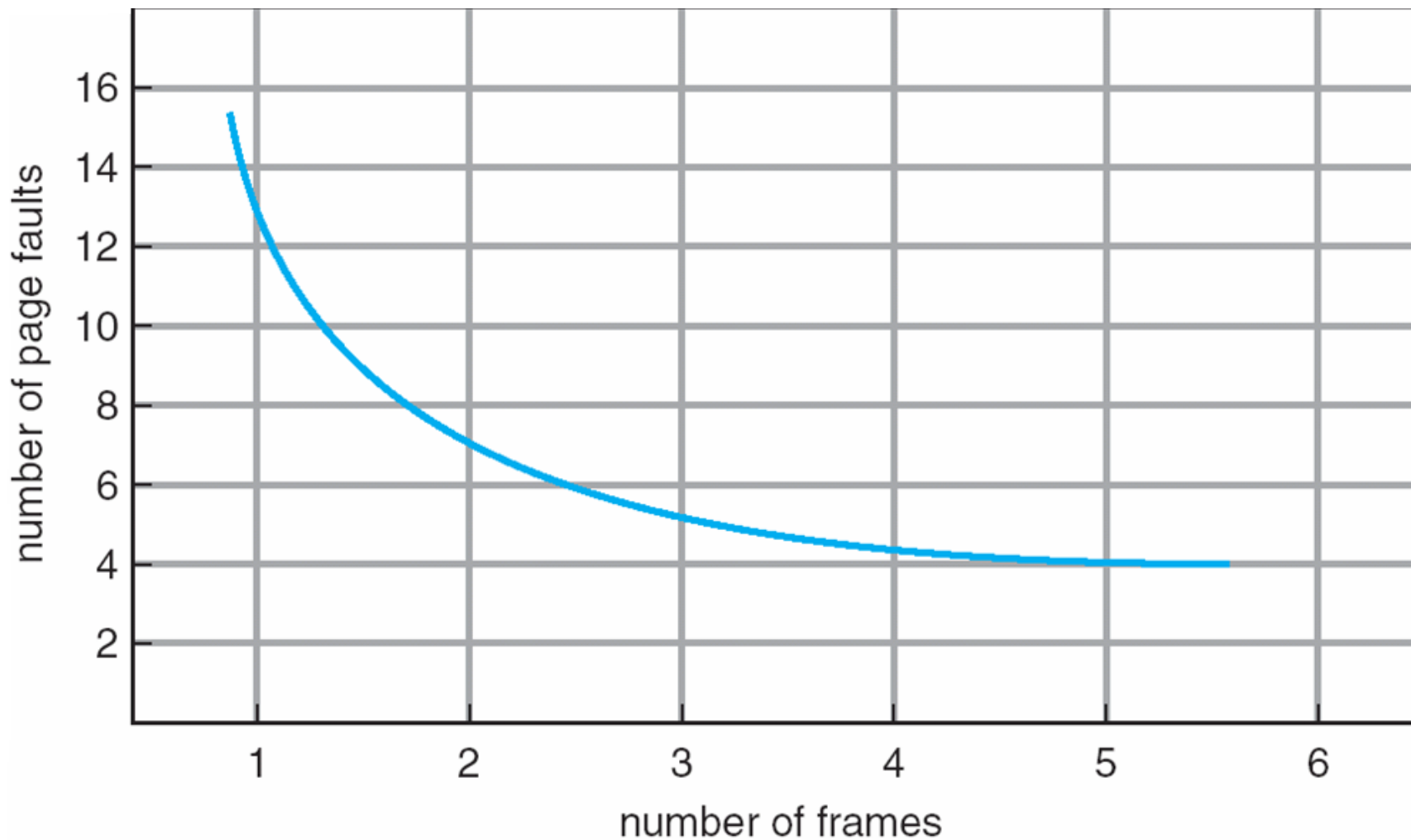
- 每个进程需要多少帧
- 要更换哪些页帧;

❖ 页面替换算法

- 希望第一次访问和重新访问的页面错误率最低;
- ❖ 通过在特定的内存引用字符串 (引用串) 上运行算法并计算该字符串上的页面错误数来评估算法;
 - 字符串只是页码, 而不是完整地址;
 - 重复访问同一页面不会导致页面错误;
 - 结果取决于可用的帧数;
- ❖ 在我们所有的示例中, 引用页码的引用字符串是
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1



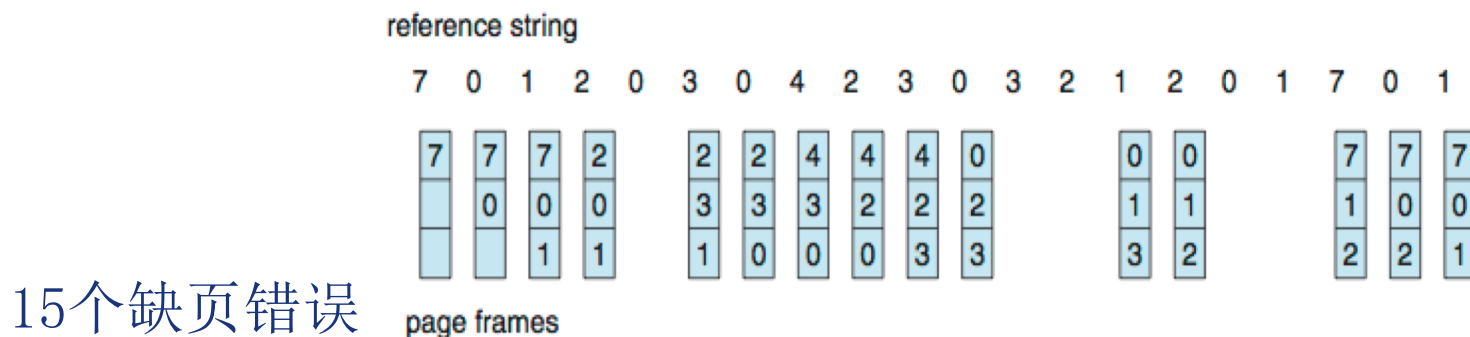
页面错误与帧数之间的关系图





先进先出FIFO算法

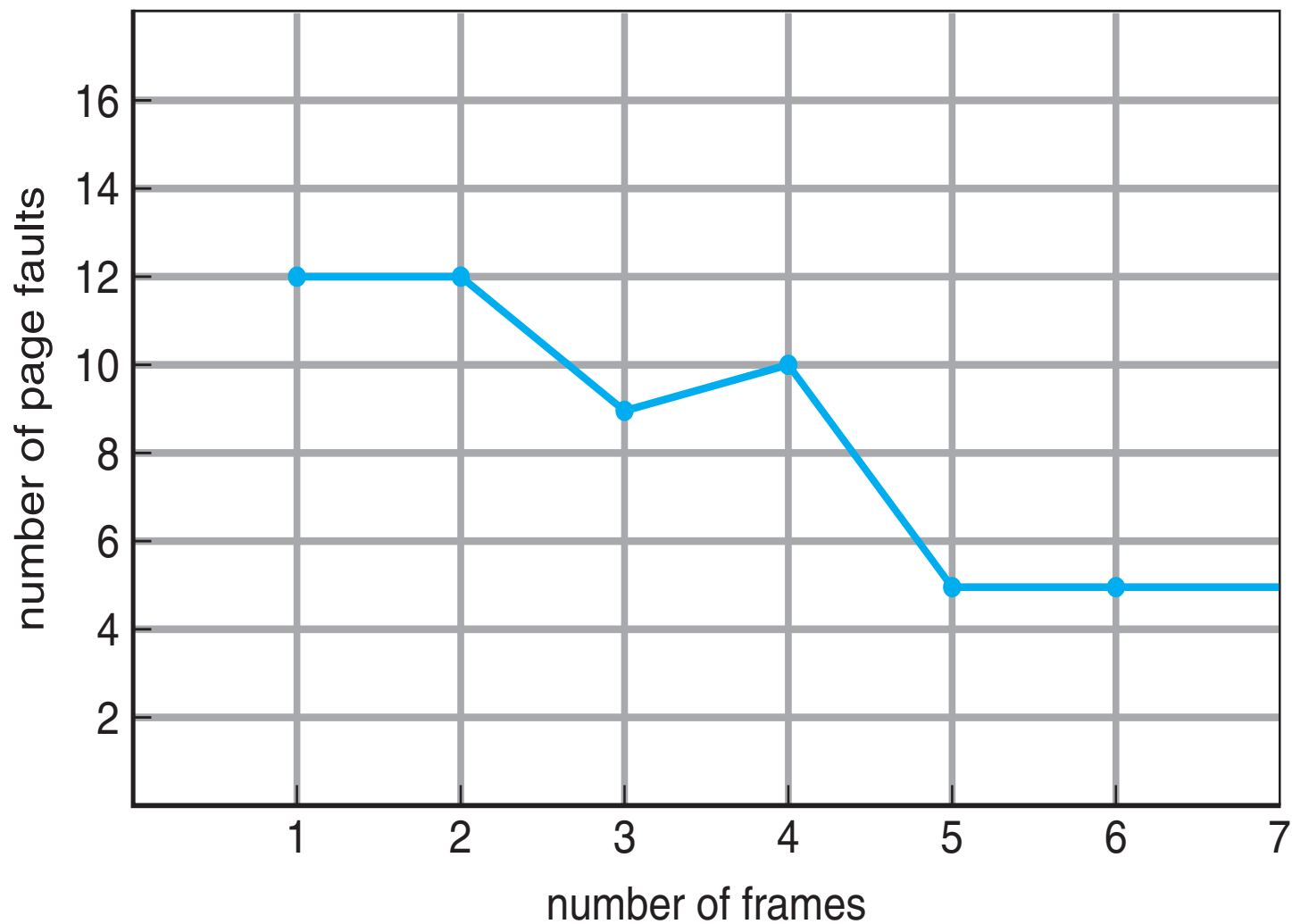
- ❖ 引用字符串: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
- ❖ 3帧 (每个进程一次可在内存中存储3页)



- ❖ 可以通过引用字符串变化: 考虑1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
 - 添加更多帧会导致更多页面错误!
 - Belady 异常;
- ❖ 如何跟踪页面的年龄?
 - 只需使用FIFO队列



FIFO解释Belady的异常



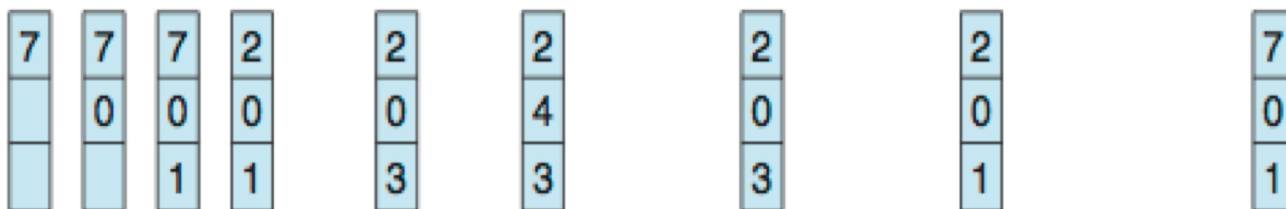


最优页面置换算法

- ❖ 替换在最长时间不使用的页面
 - 对于该示例，9个缺页错误是最佳的
- ❖ 怎么知道的？
 - 不能预测未来
- ❖ 用于测量算法的性能（比较的基线）

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames



最少最近使用 (LRU) 算法

- ❖ 使用过去的知识而不是未来的知识
- ❖ 替换在最长时间内未使用的页面
- ❖ 将最后一次使用的时间与每页关联

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0		1		1		1
	0	0	0		0		0	0	3	3		3		0		0
		1	1		3		3	2	2	2		2		2		7

page frames

- ❖ 12个缺页故障 - 比FIFO好, 但比OPT (最优置换) 差
- ❖ 一般来说, 算法很好, 经常使用
- ❖ 但如何实施呢?



LRU算法

❖ Counter implementation (计数器)

- Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
- When a page needs to be changed, look at the counters to find smallest value
 - Search through table needed

❖ Stack implementation (堆栈)

- Keep a stack of page numbers in a double link form:
- Page referenced:
 - move it to the top
 - requires 6 pointers to be changed (参见下页例子)
- But each update more expensive
- No search for replacement



LRU算法

- ❖ LRU和OPT是没有Belady异常的堆栈算法的例子
- ❖ 使用堆栈记录最近的页面引用

reference string

4 7 0 7 1 0 1 2 1 2 7 1 2

↑ ↑
a b

2
1
0
7
4

stack
before
a

7
2
1
0
4

stack
after
b



近似LRU算法

- ❖ LRU需要特殊的硬件，速度仍然很慢
- ❖ 设置引用位
 - 与每个页面关联一位，初始值=0
 - 当页面被引用时，位设置为1（由硬件实现）；
 - 通过检查引用位来确定哪些页面被使用，哪些尚未使用；
- ❖ 额外引用位算法
 - 通过定期记录引用位，可以获得额外的排序信息；
 - 为内存中的页表的每个页面保留一个8位的字节；
 - 将每个页面的引用位移到8位字节的高位，右移一位，并丢弃最低位，保存8个时间周期的页面使用情况；
 - 具有11000100的历史寄存器值的页面比具有值为01110111的页面更为“最近使用”；
 - 替换最小编号；



近似LRU算法

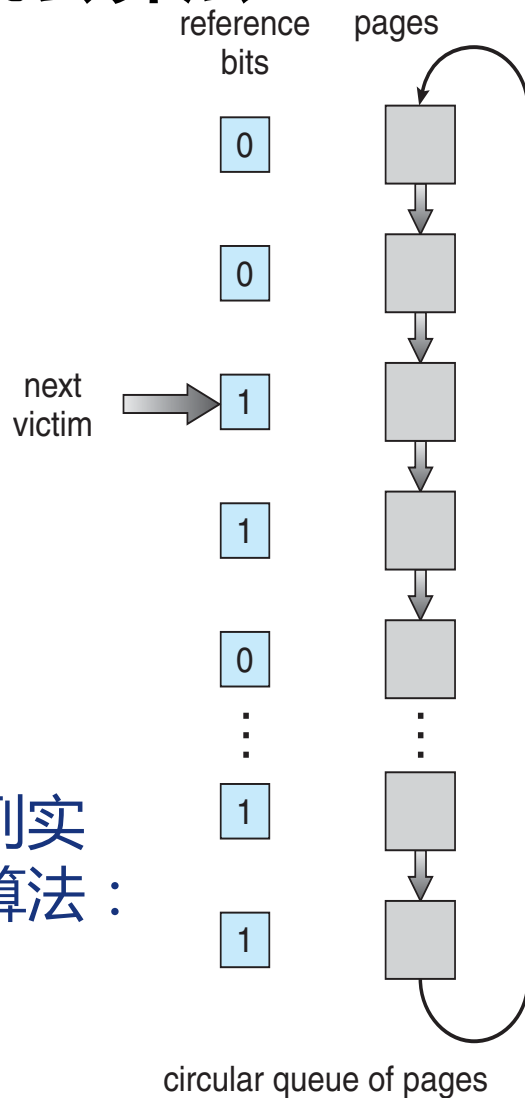
❖ 第二次机会算法

- 通常为FIFO，加上硬件提供的引用位；
- **时钟算法；**
- 如果要替换的页面已被删除
 - 引用位=0→替换它
 - 引用位=1，则：
 - 设置引用位0，将页面保留在内存中
 - 根据相同的规则替换下一页；

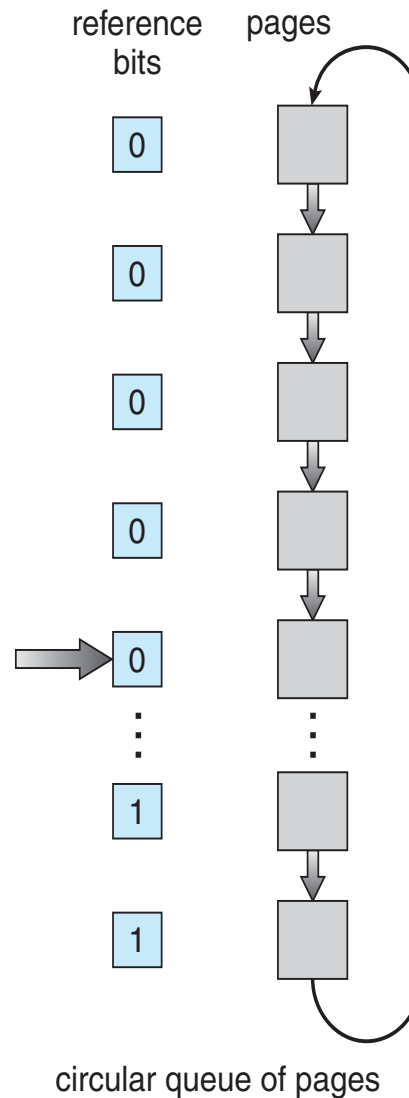


第二次机会算法

循环队列实现
时钟算法：



(a)



(b)



增强型第二次机会算法

- ❖ 通过使用引用位和**一致修改位**（如果可用）来改进算法
- ❖ 采取有序配对（引用、修改）：
 - $(0, 0)$ 最近未使用未修改 - 要替换的最佳页面
 - $(0, 1)$ 最近未使用但已修改 - 不太好，必须在置换之前将页面写出；
 - $(1, 0)$ 最近使用但没有修改 - 可能很快会再次使用
 - $(1, 1)$ 最近使用和修改的 - 可能很快会再次使用，需要在更换前将页面写出；
- ❖ 当需要页面置换时，使用时钟方案，但检查页面属于哪个类型；替换非空的最低类型中的第一个页面；
 - 可能需要多次搜索循环队列



计数算法

- ❖ 保留一个计数器，记录每页的引用次数
 - 不常见，不能很好的近似OPT；
- ❖ 最不经常使用（LFU）算法：
 - 用最小计数替换页面；
 - 定期将计数右移1位，形成指数衰减的平均计数；
- ❖ 最常使用（MFU）算法：
 - 基于这样一个论点，即计数最小的页面可能刚被引入，尚未被使用；



页面缓冲算法

❖ 始终保持一个空闲帧池

- 当出现缺页错误时，会像以前一样选择一个牺牲帧；
- 将页面读入空闲帧，选择要退出的牺牲帧并添加到空闲池；
- 方便时，驱逐牺牲帧，无需等待写出牺牲帧；

❖ 可能的话，保留修改过的页面列表

- 当备份存储为空闲时，在那里写入页面并设置为非脏；增加了无需写出的页面的概率；

❖ 可能的话，**保持空闲帧池**，并且记住哪些页面在哪些帧内；

- 如果在重用之前再次引用，则无需再次从磁盘加载内容
- 如果选择了错误的受害者帧，通常有助于减少开销和损失；



应用程序与页面置换

- ❖ 所有这些算法都让操作系统猜测未来的页面访问
- ❖ 有些应用程序具有更好的知识, 更能理解自己的内存使用与磁盘使用, 例如数据库;
- ❖ 内存密集型应用程序可能导致双缓冲
 - 操作系统将页面副本作为I/O缓冲区保存在内存中
 - 应用程序将页保留在内存中以供自己工作;
- ❖ 操作系统可以直接访问磁盘, 从而避开文件系统程序
 - 原始磁盘模式, raw disk, 产生原始I/O;
- ❖ 绕过请求调页、缓冲、锁定、预取等;
- ❖ 尽管有些应用程序在原始分区上实现自己的专用存储服务更加高效, 但是大多数应用程序采用通用文件系统服务更好;



帧分配

- ❖ 在进程之间如何分配固定数量的可用内存?
- ❖ 每个进程需要最少的帧数;
- ❖ 示例: IBM 370 - 处理SS MOVE指令的6页:
 - 指令为6字节, 可能跨越2页;
 - 处理from操作数2页;
 - 处理to操作数2页;
- ❖ 最大值当然是系统中的总帧数
- ❖ 两种主要的分配计划
 - 固定分配
 - 优先级分配
- ❖ 许多变体



固定分配

- ❖ 相等分配 - 例如, 如果有100帧 (在为操作系统分配帧之后) 和5个进程, 则为每个进程分配20帧
 - 保留一些作为空闲帧缓冲池

- ❖ 比例分配 - 根据进程的大小进行分配

- 随着多道程序设计程度的提高, 进程大小也会发生变化
- 根据优先级分配;

s_i = size of process p_i

$$S = \sum s_i$$

m = total number of frames

$$a_i = \text{allocation for } p_i = \frac{s_i}{S} \times m$$

$$m = 64$$

$$s_1 = 10$$

$$s_2 = 127$$

$$a_1 = \frac{10}{137} \times 62 \approx 4$$

$$a_2 = \frac{127}{137} \times 62 \approx 57$$



全局分配与局部分配

- ❖ 全局置换 - 进程从所有帧集中选择替换帧；一个进程可以从另一个进程获取帧
 - 但是，进程执行时间可能会有很大的变化
 - 但具有更大的吞吐量更常见
 - 高优先级进程可以置换低优先级进程；
 - 缺点：进程不能控制他自己的缺页错误率；
- ❖ 局部置换 - 每个进程仅从其自己的一组分配帧中进行选择
 - 每个进程的性能更加一致；
 - 但可能是未充分利用的内存；

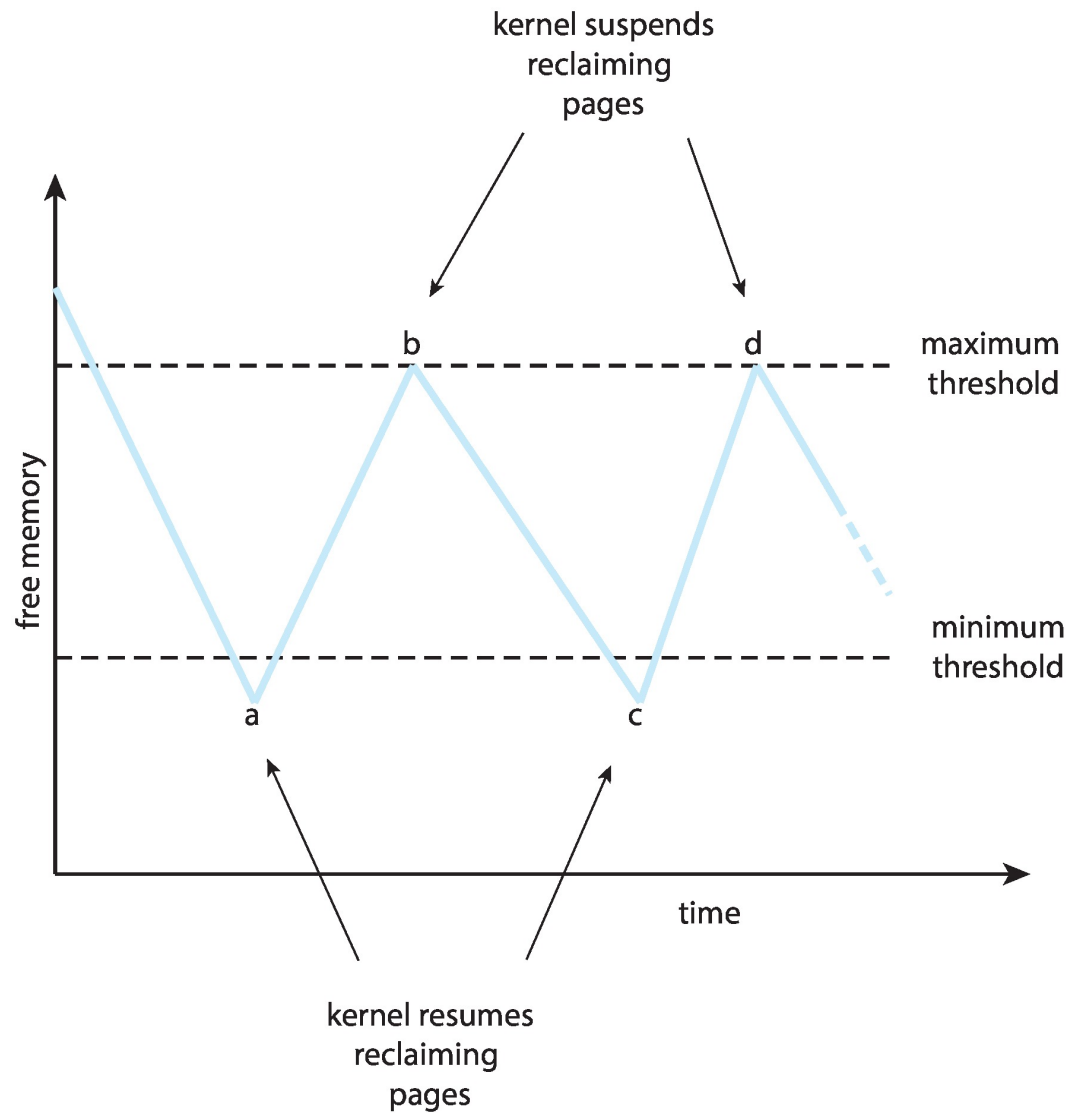


页面回收

- ❖ 实施全局页面替换算法的策略;
- ❖ 所有的内存请求都从**空闲帧列表**中得到满足, 而不是在开始选择要替换的页面之前等待列表降到零;
- ❖ 当空闲帧列表低于某个阈值时, 将触发页面替换。此策略试图确保始终有足够的可用内存来满足新的请求。



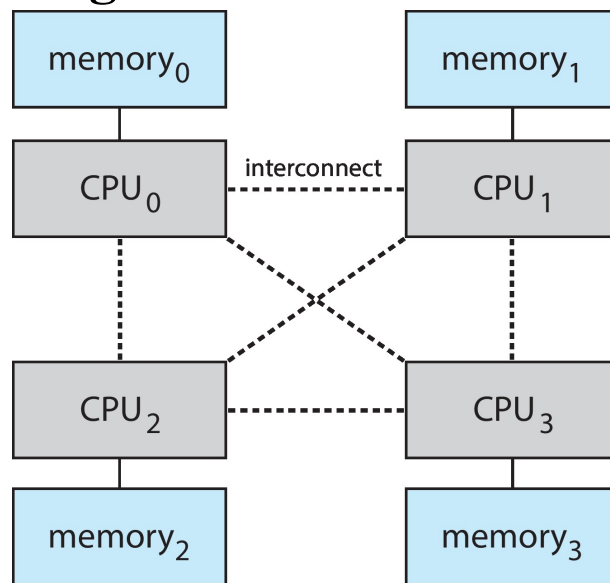
页面回收





非均匀内存访问 (Non-uniform memory access, NUMA)

- ❖ So far, we assumed that all memory accessed equally
- ❖ Many systems are NUMA – speed of access to memory varies
 - Consider system boards containing CPUs and memory, interconnected over a system bus
- ❖ NUMA multiprocessing architecture





非均匀内存访问 (Non-uniform memory access, NUMA)

❖ **Optimal performance comes from allocating memory “close to” the CPU on which the thread is scheduled**

- And modifying the scheduler to schedule the thread on the same system board when possible
- Solved by Solaris by creating **lggroups**
 - Structure to track CPU / Memory low latency groups
 - Used my schedule and pager
 - When possible schedule all threads of a process and allocate all memory for that process within the lggroup



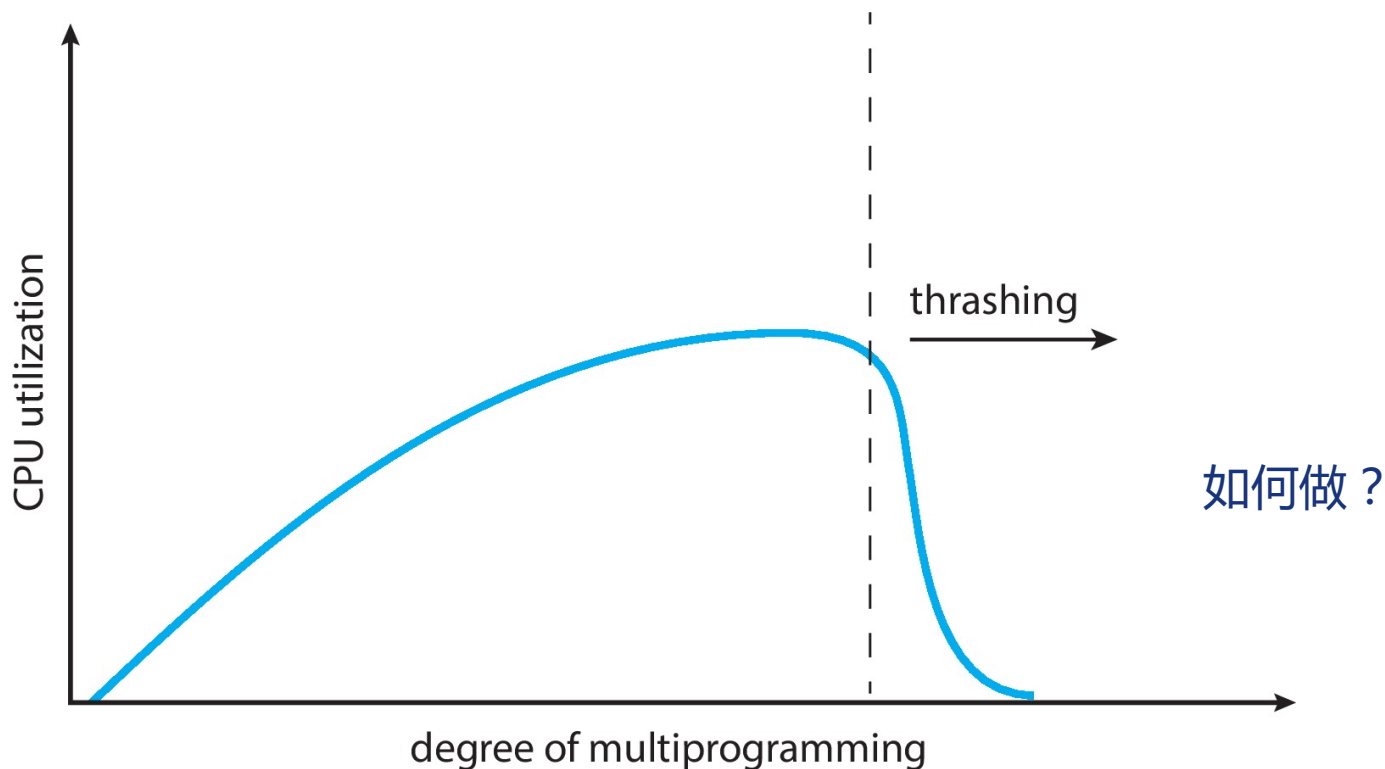
系统抖动

- ❖ If a process does not have “enough” pages, the page-fault rate is very high
 - Page fault to get page
 - Replace existing frame
 - But quickly need replaced frame back
 - This leads to:
 - Low CPU utilization
 - **Operating system thinking that it needs to increase the degree of multiprogramming**
 - **Another process added to the system**



系统抖动

- ❖ **Thrashing.** A process is busy swapping pages in and out
- 进程的调页时间多于执行时间，那么这个进程就在抖动





系统抖动

❖ 为什么请求分页能够有效工作？

局部性模型

- 局部性是最近使用页面的一个集合；
- 进程从一个局部移向另一个局部；
- 不同的局部之间存在交集；

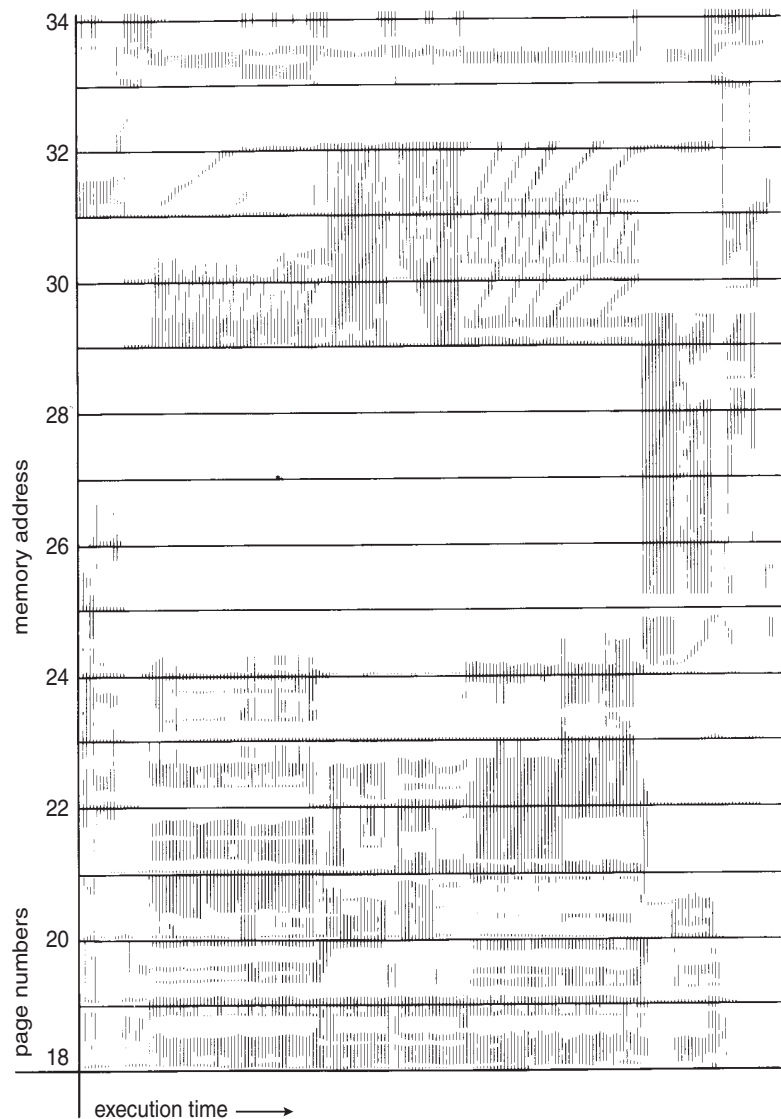
❖ 为什么抖动会发生？

$$\Sigma \text{ size of locality} > \text{total memory size}$$

❖ 通过使用本地或优先级页面替换来限制系统抖动影响



时空局部性





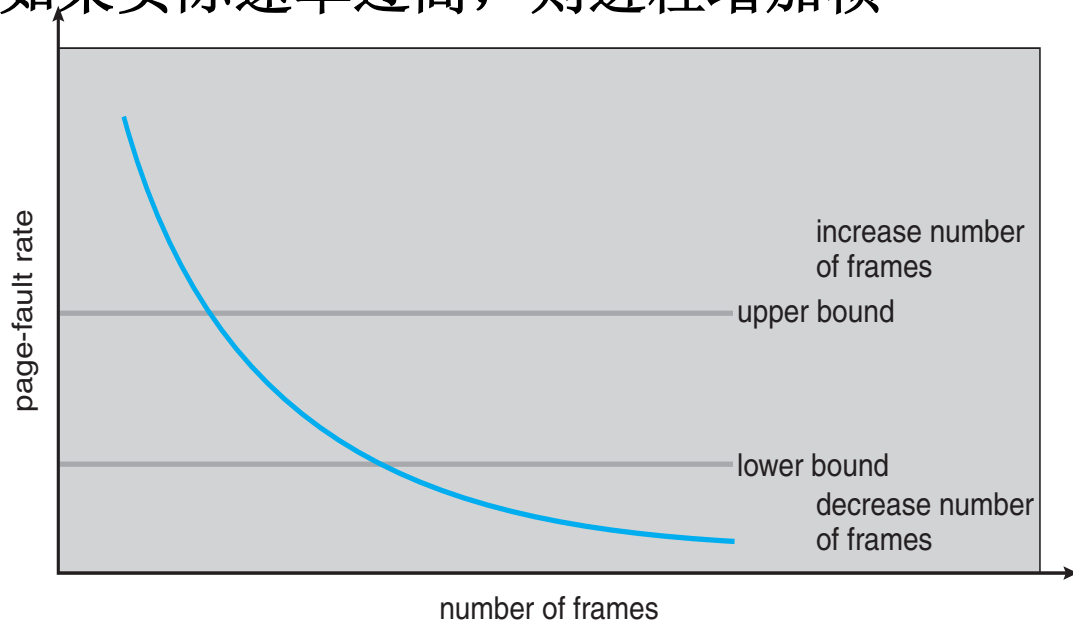
工作集 (Working set)

- ❖ 工作集是基于局部性假设;
- ❖ $\Delta \equiv \text{working-set window} \equiv \text{a fixed number of page references}$
Example: 10,000 instructions
- ❖ WSS_i (working set of Process P_i) = total number of pages referenced in the most recent Δ (varies in time)
 - if Δ too small will not encompass entire locality
 - if Δ too large will encompass several localities
 - if $\Delta = \infty \Rightarrow$ will encompass entire program
- ❖ $D = \sum WSS_i \equiv \text{total demand frames}$
 - Approximation of locality



缺页错误率

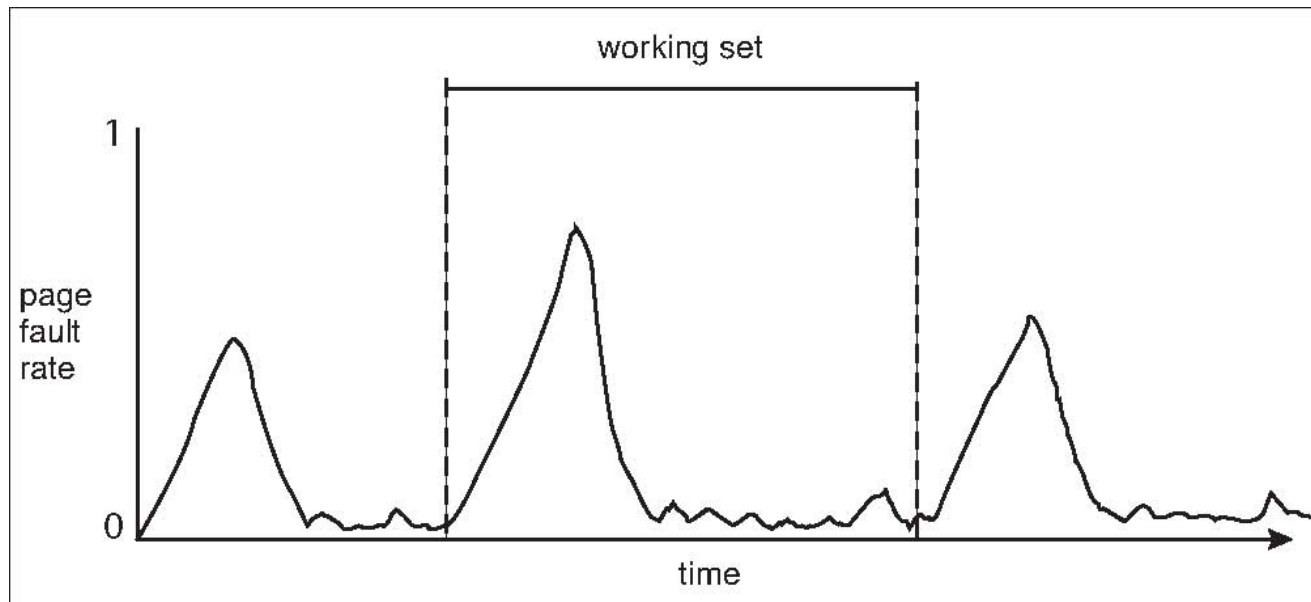
- ❖ 比工作集更加直接的方法;
- ❖ 用于防止抖动;
- ❖ 建立“可接受”缺页故障频率 (PFF) 并使用本地替换策略;
 - 如果实际速率过低, 进程将释放帧;
 - 如果实际速率过高, 则进程增加帧





工作集与缺页错误率

- 进程的工作集与其页面故障率之间有直接关系
- 工作集随时间变化
- 随时间变化的波峰和波谷





分配内核内存

- ❖ 与用户内存不同的处理方式
- ❖ 通常从可用内存池中分配
 - 内核为不同大小的结构请求内存
 - 某些内核内存需求是连续的
 - 即，用于设备I/O;



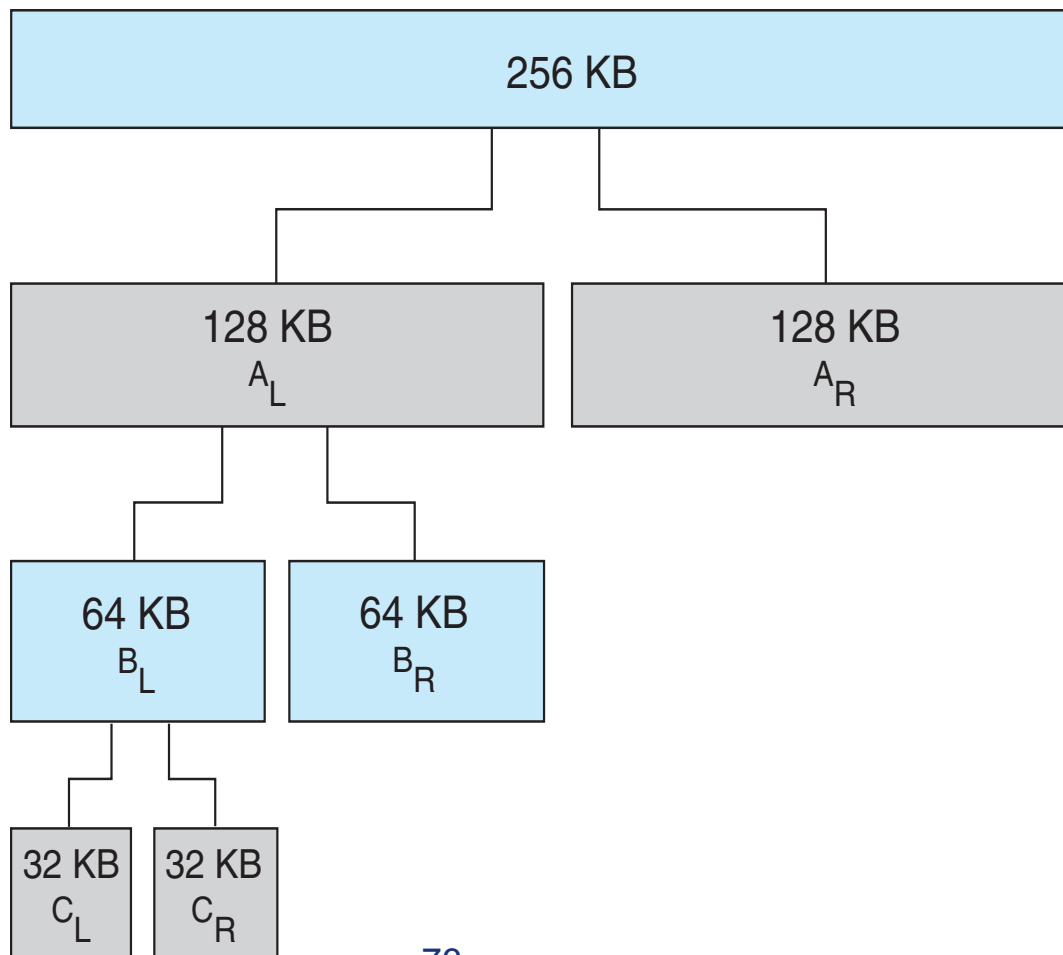
伙伴系统

- ❖ **Allocates memory from fixed-size segment consisting of physically-contiguous pages**
- ❖ **Memory allocated using power-of-2 allocator**
 - Satisfies requests in units sized as power of 2
 - Request rounded up to next highest power of 2
 - When smaller allocation needed than is available, current chunk split into two buddies of next-lower power of 2
 - Continue until appropriate sized chunk available
- ❖ **For example, assume 256KB chunk available, kernel requests 21KB**
 - Split into A_L and A_R of 128KB each
 - One further divided into B_L and B_R of 64KB
 - One further into C_L and C_R of 32KB each – one used to satisfy request
- ❖ **Advantage – quickly coalesce (合并) unused chunks into larger chunk**
- ❖ **Disadvantage – fragmentation (碎片)**



伙伴系统分配器

physically contiguous pages



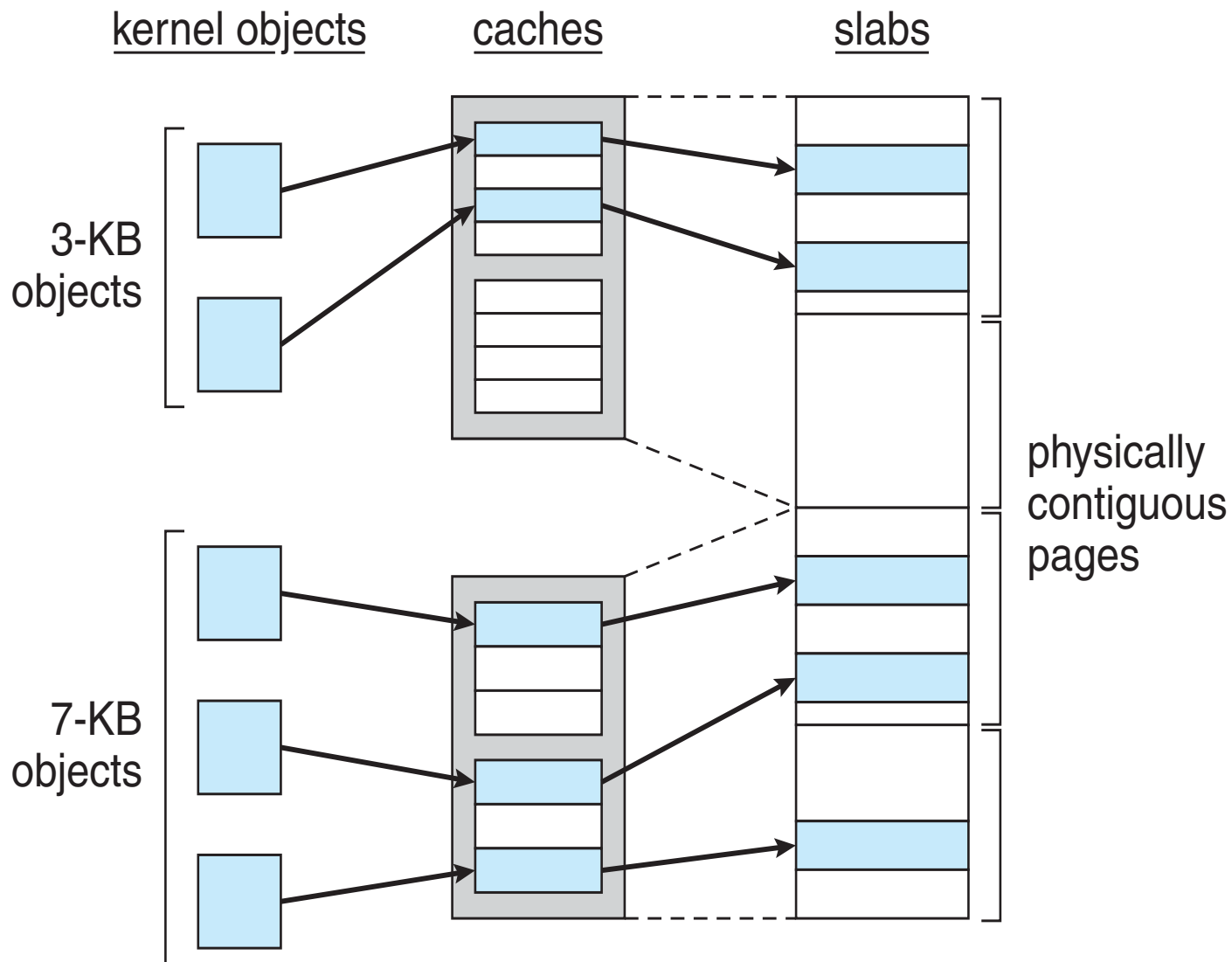


Slab分配器

- ❖ Alternate strategy (替代策略)
- ❖ Slab is one or more physically contiguous pages
- ❖ **Cache** consists of one or more slabs
- ❖ Single cache for each **unique kernel data structure**
 - Each cache filled with **objects** – instantiations of the data structure
- ❖ When cache created, filled with objects marked as free
- ❖ When structures stored, objects marked as used
- ❖ If one slab is full of used objects, next object allocated from empty slab
 - If no empty slabs, new slab allocated
- ❖ Benefits include **no fragmentation**, fast memory request satisfaction



Slab分配器





Linux中的Slab分配器

- ❖ For example **process descriptor** is of type struct task_struct
- ❖ Approx 1.7KB of memory
- ❖ New task -> allocate new struct from cache
 - Will use existing free struct task_struct
- ❖ Slab can be in three possible states
 1. Full – all used
 2. Empty – all free
 3. Partial – mix of free and used
- ❖ Upon request, slab allocator
 1. Uses free struct in partial slab
 2. If none, takes one from empty slab
 3. If no empty slab, create new empty

Slab的主要优点： 没有因碎片而引起内存浪费；可以快速满足内存需求；



Linux中的Slab分配器

- ❖ Slab started in Solaris, now wide-spread for both kernel mode and user memory in various OSes
- ❖ Linux 2.2 had SLAB, now has both SLOB and SLUB allocators
 - SLOB for systems with limited memory
 - Simple List of Blocks – maintains 3 list objects for small, medium, large objects
 - SLUB is performance-optimized SLAB removes per-CPU queues, metadata stored in page structure



其他注意事项

- ❖ **Prepaging**
- ❖ **Page size**
- ❖ **TLB reach**
- ❖ **Inverted page table**
- ❖ **Program structure**
- ❖ **I/O interlock and page locking**



预调页

- ❖ To reduce the large number of page faults that occurs at process startup;
- ❖ Prepage all or some of the pages a process will need, before they are referenced;
- ❖ But if prepaged pages are unused, I/O and memory was wasted
- ❖ Assume s pages are prepaged and α of the pages is used
 - Is cost of $s * \alpha$ save pages faults $>$ or $<$ than the cost of prepaging $s * (1 - \alpha)$ unnecessary pages?
 - α near zero $\Rightarrow$ prepaging loses



页面大小

- ❖ **Sometimes OS designers have a choice**
 - Especially if running on custom-built CPU
- ❖ **Page size selection must take into consideration:**
 - Fragmentation
 - Page table size
 - **Resolution**
 - I/O overhead
 - Number of page faults
 - Locality
 - TLB size and effectiveness
- ❖ **Always power of 2, usually in the range 2^{12} (4,096 bytes) to 2^{22} (4,194,304 bytes)**
- ❖ **On average, growing over time**



TLB范围

- ❖ **TLB Reach - The amount of memory accessible from the TLB**
- ❖ **TLB Reach = (TLB Size) X (Page Size)**
- ❖ **Ideally, the working set of each process is stored in the TLB**
 - Otherwise there is a high degree of page faults
- ❖ **Increase the Page Size**
 - This may lead to an increase in fragmentation as not all applications require a large page size;
- ❖ **Provide Multiple Page Sizes**
 - This allows applications that require larger page sizes the opportunity to use them without an increase in fragmentation



程序结构

❖ Program structure

- `int[128,128] data;`
- Each row is stored in one page
- Program 1

```
for (j = 0; j < 128; j++)
    for (i = 0; i < 128; i++)
        data[i,j] = 0;
```

$128 \times 128 = 16,384$ page faults

- Program 2

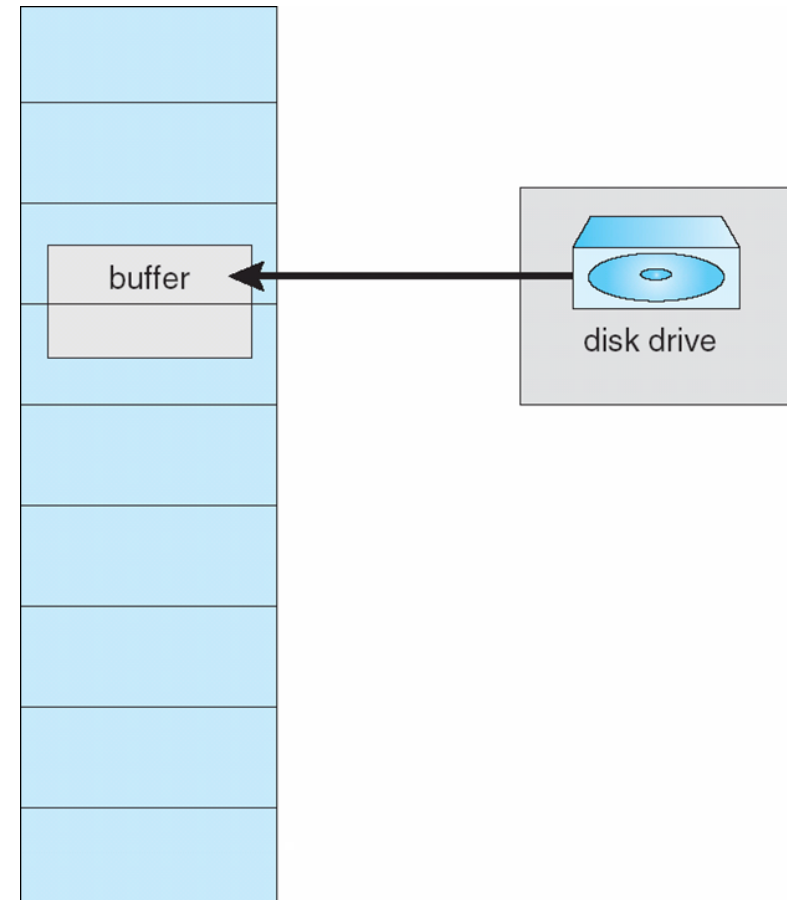
```
for (i = 0; i < 128; i++)
    for (j = 0; j < 128; j++)
        data[i,j] = 0;
```

128 page faults



I/O联锁

- ❖ I/O Interlock – **Pages must sometimes be locked into memory**
- ❖ Consider I/O - Pages that are used for copying a file from a device must be locked from being selected for eviction by a page replacement algorithm
- ❖ Pinning of pages to lock into memory





Windows

- ❖ Uses demand paging with **clustering**. Clustering brings in pages surrounding the faulting page
- ❖ Processes are assigned **working set minimum and working set maximum**
- ❖ Working set minimum is the minimum number of pages the process is guaranteed to have in memory
- ❖ A process may be assigned as many pages up to its working set maximum
- ❖ When the amount of free memory in the system falls below a threshold, automatic working set trimming is performed to restore the amount of free memory
- ❖ Working set trimming removes pages from processes that have pages in excess of their working set minimum

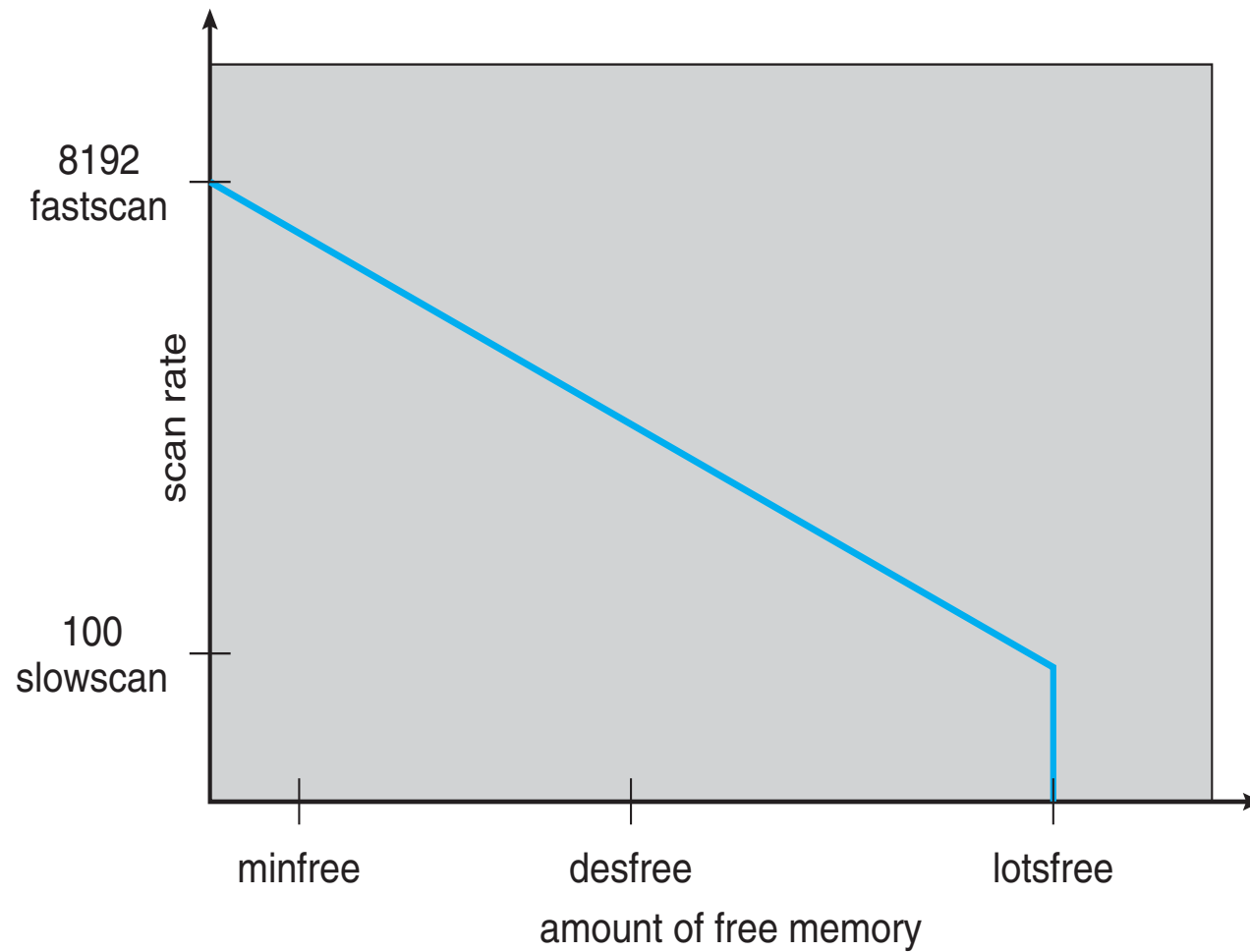


Solaris

- ❖ Maintains a list of free pages to assign faulting processes
- ❖ Lotsfree – threshold parameter (amount of free memory) to begin paging
- ❖ Desfree – threshold parameter to increasing paging
- ❖ Minfree – threshold parameter to being swapping
- ❖ Paging is performed by pageout process
- ❖ Pageout scans pages using modified clock algorithm
- ❖ Scanrate is the rate at which pages are scanned. This ranges from slowscan to fastscan
- ❖ Pageout is called more frequently depending upon the amount of free memory available
- ❖ *Priority paging* gives priority to process code pages



Solaris 页面扫描





中山大學
SUN YAT-SEN UNIVERSITY

计算机学院（软件学院）

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



谢谢